OOPS Scenario based questions

Let me share something with you, one-to-one.

When it comes to **OOPs**, most people think they've got it covered. They can define encapsulation, they can list the four pillars, they can even explain inheritance on paper. But the moment an interviewer says: *'Okay, show me how you'd use this in a real project,'* that's where the silence begins.

And trust me, I've seen this happen again and again. It's not that candidates don't know the theory, it's that they've never practiced applying it. That's why I created these OOPs scenario-based questions**.**

Here's how I want you to treat them:

- Imagine you're in front of me in a live mock interview.
- Don't just write code; first, explain your thinking in plain English. Why a class here? Why an interface there? Why aggregation instead of inheritance?
- As you go through each scenario, notice how I've written not just the code but the *reasoning*. That's the gold. That's the part interviewers actually care about.
- Once you're comfortable, start tweaking things. Change inputs, add another class, refactor the relationships. You'll realize how naturally OOPs concepts start flowing when you practice this way.

If you take these exercises seriously, you'll stop sounding like someone reciting a textbook. Instead, you'll come across as someone who can think in objects, who can design systems, and who can *explain it clearly* to anyone in the room. And that's exactly the kind of candidate who gets a nod of approval in interviews.

So treat these as our personal mock interview sessions, me giving you real-world prompts, and you practicing your step-by-step answers until they sound confident and natural.

Let's dive in.

#	Scenario (Short)	Topic / Concept	Level	Role Focus
1	Design Staff class to store records and prevent duplicates	Classes & Objects	L2	Backend Core
2	Represent books uniquely using ISBN in a library	Classes & Objects	L2	Backend Core
3	Handle product price & stock updates safely	Classes & Objects	L3	Backend Core
4	Design Restaurant class with name, cuisine, and rating	Classes & Objects	L1	Backend Core
5	Model Ticket & Passenger using composition	Classes & Objects	L2	Backend Core
6	Auto-assign tracking ID in default constructor	Constructors	L2	Backend Core
7	Enforce mandatory title & price using parameterized constructor	Constructors	L2	Backend Core
8	Duplicate a Ticket object for another passenger (copy constructor)	Constructors	L2	Backend Core
9	Avoid Car constructor duplication using constructor chaining	Constructors	L2	Backend Core
10	Restrict ConfigManager instantiation with private constructor	Constructors / Singleton	L3	Advanced Backend
11	Model Transport → Truck, Scooter (single inheritance)	Inheritance	L2	Backend Core
12	Model Person → Teacher → Professor (multilevel inheritance)	Inheritance	L2	Backend Core
13	Represent MedProfessional → Cardiologist, Orthopedist	Inheritance	L2	Backend Core
14	Extend Employee → Manager to avoid code duplication	Inheritance	L2	Backend Core

15	Model Account → Savings → HighInterestSavings	Inheritance	L3	Backend Core
16	Implement SmartPhone as Camera + MusicPlayer via interfaces	Multiple Inheritance / Interface	L3	Advanced Backend
17	Overload add() in MathTool for multiple signatures	Method Overloading	L1	Backend Core
18	Overload discounts for single vs multiple products	Method Overloading	L2	Backend Core
19	Overload transfer() by account or phone number	Method Overloading	L2	Backend Core
20	Overload sendMessage() for text, image, video	Method Overloading	L2	Backend Core
21	Overload bookTicket() for passenger ID vs passport	Method Overloading	L2	Backend Core
22	Override calculateSalary() for full-time vs part-time	Method Overriding	L2	Backend Core
23	Override showEnrollmentInfo() for Regular vs Flexible	Method Overriding	L2	Backend Core
24	Override computeCost() in Finance vs Logistics	Method Overriding	L3	Backend Core
25	Override calculateBenefits() for Contract vs Permanent	Method Overriding	L2	Backend Core
26	Override computeTax() for Individual, Company, NGO	Method Overriding	L3	Backend Core
27	Apply all access modifiers in Digital Library	Access Modifiers	L2	Backend Core
28	Restrict salary (private) and expose name (public)	Access Modifiers	L1	Backend Core
29	Protect SSN, expose diagnosis to subclass	Access Modifiers	L2	Backend Core
30	Keep quiz answers private but scores public	Access Modifiers	L2	Backend Core

31	Restrict appraisal visibility to HR package	Access Modifiers	L3	Backend Core
32	Use static vs instance variables in School	Static & Instance	L2	Backend Core
33	Static counter for total students	Static	L2	Backend Core
34	Represent shared device state using static fields	Static	L3	Backend Core
35	Static method to generate courier tracking IDs	Static Method	L3	Backend Core
36	Train number instance, static railway zone	Static & Instance	L2	Backend Core
37	Use static leaderboard across game sessions	Static / Advanced	L3	Full Stack
38	Encapsulate bank balance; expose secure getter	Encapsulation	L3	Backend Core
39	Enforce immutability for confidential medical data	Encapsulation	L3	Advanced Backend
40	Hide payment details, show masked card	Encapsulation	L2	Backend Core
41	Allow premium update only after verification	Encapsulation	L3	Backend Core
42	Secure ATM PIN update using OTP	Encapsulation / Validation	L3	Backend Core
43	Abstract class for pricing virtual vs classroom course	Abstraction	L3	Backend Core
44	Abstract base for fare formulas (Coach, Aircraft)	Abstraction	L3	Backend Core
45	Payment interface for UPI, Card, NetBanking	Abstraction / Interface	L3	Backend Core
46	Abstract contract for CarRide, BikeRide, AutoRide	Abstraction	L3	Backend Core

47	Common parse() for CSV, XML, JSON	Abstraction	L3	Backend Core
48	Overloading multiply() shows compile-time polymorphism	Polymorphism	L2	Backend Core
49	Runtime polymorphism for Regular vs Premium playContent()	Polymorphism	L3	Backend Core
50	Interface-based poly in Finance & Logistics modules	Polymorphism	L3	Backend Core
51	Characters (Warrior, Archer, Mage) override attack()	Polymorphism	L3	Backend Core
52	Polymorphism: Doctor, Nurse, Surgeon performDuty()	Polymorphism	L3	Backend Core
53	Payment methods (Card, Wallet, COD) runtime polymorphism	Polymorphism	L3	Backend Core
54	Enforce max subjects per semester using final	Final Keyword	L2	Backend Core
55	Define universal GST constant	Final / Static Final	L2	Backend Core
56	Prevent subclassing in UserAccount	Final Class	L3	Advanced Backend
57	Resolve variable shadowing with this	Constructor / this	L2	Backend Core

Classes and Objects

Scenario 1: Staff records creation

A company wants to store staff details (name, code, team) and allow creation of multiple staff records.

Answer:

I'll start with an object to represent each staff member.

- Name and team are Strings; code is a String that uniquely identifies a staff member.
- I'll keep fields private and validate in the constructor so invalid data doesn't enter the system.
- I'll implement equals/hashCode using code to prevent duplicates in a Set.

Code:

```
import java.util.*;

class Staff {
   private final String name;
   private final String code;  // unique identity
   private final String team;

   public Staff(String name, String code, String team) {
      if (name == null || name.trim().isEmpty()) throw new
IllegalArgumentException("Name required");
      if (code == null || code.trim().isEmpty()) throw new
IllegalArgumentException("Code required");
      this.name = name.trim();
      this.code = code.trim();
      this.team = (team == null || team.trim().isEmpty()) ? "Unassigned" : team.trim();
   }

   public String getName() { return name; }
   public String getCode() { return code; }
   public String getTeam() { return team; }

   @Override public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Staff)) return false;
      return code.equals(((Staff) o).code);
   }
   @Override public int hashCode() { return Objects.hash(code); }

   @Override public String toString() {
      return "%s (%s) - Team: %s".formatted(name, code, team);
```

```
  }
}
```

```
class StaffDemo {
  public static void main(String[] args) {
    Set<Staff> staff = new HashSet<>();
    staff.add(new Staff("Rajat", "E101", "Platform"));
    staff.add(new Staff("Anita", "E102", "Data"));
    staff.add(new Staff("Rajat", "E101", "Platform")); // duplicate by code; Set blocks
it
    staff.forEach(System.out::println);
  }
}
```

Reasoning to interviewer:

- Private fields + validation in the constructor enforce invariants (encapsulation).
- code is the natural identity, so equals/hashCode are based on it, enabling Set/Map usage.
- Team defaults to “Unassigned” to avoid null checks across the codebase.

Future scope:

- Add repository/service layers (search by team, pagination).
- Add builder pattern if construction grows.
- Persist to DB with a unique index on code.

Scenario 2: Book representation in a library

A library management system needs to represent each book with a title, author, and ISBN.

Answer:
Each book is an object.

- Title, author, isbn are Strings. ISBN uniquely identifies a book, so I'll normalize it and use it in equality.
- I'll keep the class mostly immutable to avoid accidental changes.

Code:

```
import java.util.*;

class Book {
   private final String title;
   private final String author;
   private final String isbn; // unique identity

   public Book(String title, String author, String isbn) {
      if (title == null || title.trim().isEmpty()) throw new IllegalArgumentException("Title
required");
      if (author == null || author.trim().isEmpty()) throw new
IllegalArgumentException("Author required");
      if (isbn == null || !isbn.matches("[0-9Xx-]{10,17}")) throw new
IllegalArgumentException("Invalid ISBN-like value");
      this.title = title.trim();
      this.author = author.trim();
      this.isbn = isbn.replaceAll("-", "").toUpperCase(); // normalize
   }

   public String getTitle()  { return title; }
   public String getAuthor() { return author; }
   public String getIsbn()   { return isbn; }

   @Override public boolean equals(Object o) {
      return (o instanceof Book b) && isbn.equals(b.isbn);
   }
   @Override public int hashCode() { return Objects.hash(isbn); }

   @Override public String toString() {
      return "\"%s\" by %s [ISBN:%s]".formatted(title, author, isbn);
   }
}
```

```
class LibraryDemo {
   public static void main(String[] args) {
      Set<Book> catalog = new HashSet<>();
      catalog.add(new Book("Clean Code", "Robert C. Martin", "978-0132350884"));
      catalog.add(new Book("Effective Java", "Joshua Bloch", "9780134685991"));
      catalog.forEach(System.out::println);
   }
}
```

Reasoning to interviewer:

- Immutability reduces bugs and makes objects safe to share across threads.
- ISBN normalization prevents duplicates with different dash patterns.
- equals/hashCode based on ISBN enables fast lookups in HashSet/HashMap.

Future scope:

- Add proper ISBN-10/13 checksum validation.
- Track copies/availability separately from the Book metadata.
- Introduce a DTO for API responses to avoid leaking internal structure.

Scenario 3: E-commerce product with price and stock

An e-commerce website needs a Product class to store price, stock, and description. Customers should be able to create product objects and display details.

Answer:

I'll use BigDecimal for price to avoid floating-point errors and int for stock.

- Validate description, non-negative price and stock.
- Provide methods to safely update stock and price.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

class Product {
   private final String description;
   private BigDecimal price; // currency-safe
   private int stock;

   public Product(String description, BigDecimal price, int stock) {
      if (description == null || description.trim().isEmpty()) throw new
IllegalArgumentException("Description required");
      if (price == null || price.signum() < 0) throw new IllegalArgumentException("Price
must be >= 0");
      if (stock < 0) throw new IllegalArgumentException("Stock must be >= 0");
      this.description = description.trim();
      this.price = price.setScale(2, RoundingMode.HALF_UP);
      this.stock = stock;
   }

   public synchronized void addStock(int qty) {
      if (qty <= 0) throw new IllegalArgumentException("Quantity must be > 0");
      stock += qty;
   }

   public synchronized void removeStock(int qty) {
      if (qty <= 0 || qty > stock) throw new IllegalArgumentException("Invalid quantity");
      stock -= qty;
   }

   public synchronized void updatePrice(BigDecimal newPrice) {
      if (newPrice == null || newPrice.signum() < 0) throw new
IllegalArgumentException("Invalid price");
      price = newPrice.setScale(2, RoundingMode.HALF_UP);
   }
```

```
  @Override public String toString() {
    return "%s | ₹%s | stock: %d".formatted(description, price, stock);
  }
}

class ProductDemo {
  public static void main(String[] args) {
    Product p = new Product("Noise-cancelling Headphones", new
BigDecimal("4999.50"), 12);
    System.out.println(p);
    p.addStock(3);
    p.removeStock(2);
    p.updatePrice(new BigDecimal("4799.00"));
    System.out.println(p);
  }
}
```

Reasoning to interviewer:

- BigDecimal avoids currency precision bugs.
- Synchronized mutators keep stock consistent if multiple orders hit concurrently.
- Validation protects against negative stock/price.

Future scope:

- Add currency code and price history.
- Extract inventory logic to a separate service; apply optimistic locking when persisted.

Scenario 4: Restaurants as objects in a food-delivery app

An online food delivery app needs to represent restaurants as objects, each holding name, cuisine type, and rating.

Answer:

I'll model a Restaurant with name, a controlled Cuisine enum, and a rating between 0.0 and 5.0.

- Validate and normalize inputs; round rating to one decimal place for display consistency.

Code:

```
enum Cuisine { INDIAN, CHINESE, ITALIAN, MEXICAN, THAI, JAPANESE, AMERICAN, OTHER }

class Restaurant {
  private final String name;
  private final Cuisine cuisine;
  private final double rating; // 0.0 to 5.0

  public Restaurant(String name, Cuisine cuisine, double rating) {
    if (name == null || name.trim().isEmpty()) throw new IllegalArgumentException("Name required");
    if (cuisine == null) throw new IllegalArgumentException("Cuisine required");
    if (rating < 0.0 || rating > 5.0) throw new IllegalArgumentException("Rating must be between 0 and 5");
    this.name = name.trim();
    this.cuisine = cuisine;
    this.rating = Math.round(rating * 10.0) / 10.0; // 1 decimal
  }

  public String getName()   { return name; }
  public Cuisine getCuisine(){ return cuisine; }
  public double getRating() { return rating; }

  @Override public String toString() {
    return "%s | %s | ★ %.1f".formatted(name, cuisine, rating);
  }
}

class RestaurantDemo {
  public static void main(String[] args) {
    Restaurant r = new Restaurant("Spice Route", Cuisine.INDIAN, 4.36);
    System.out.println(r); // Spice Route | INDIAN | ★ 4.4
  }
}
```

Reasoning to interviewer:

- Enum prevents typos like “Indain”.
- Keeping rating as a primitive double is fine for display; if we needed precise aggregation, we’d store raw votes and compute average separately.
- toString gives a clean display format the UI can reuse.

Future scope:

- Add location coordinates, delivery radius, and open/close times.
- Introduce a Review entity and compute rating from reviews.

Scenario 5: Ticket objects in a booking system

In a ticket-booking system, each ticket is an object with seat number, passenger details, and fare.

Answer:

I'll use composition: Ticket has a Passenger object.

- seatNo is a String to handle various formats (12A, B04).
- fare uses BigDecimal for currency accuracy.

Code:

```
import java.math.BigDecimal;
import java.util.Objects;

class Passenger {
   private final String name;
   private final String contact;

   public Passenger(String name, String contact) {
      if (name == null || name.trim().isEmpty()) throw new
IllegalArgumentException("Passenger name required");
      this.name = name.trim();
      this.contact = (contact == null) ? "" : contact.trim();
   }

   public String getName() { return name; }
   public String getContact() { return contact; }
}

class Ticket {
   private final String seatNo;
   private final Passenger passenger;
   private final BigDecimal fare;

   public Ticket(String seatNo, Passenger passenger, BigDecimal fare) {
      if (seatNo == null || seatNo.trim().isEmpty()) throw new
IllegalArgumentException("Seat required");
      if (passenger == null) throw new IllegalArgumentException("Passenger required");
      if (fare == null || fare.signum() < 0) throw new IllegalArgumentException("Invalid
fare");
      this.seatNo = seatNo.trim();
      this.passenger = passenger;
      this.fare = fare;
   }

   public String getSeatNo()     { return seatNo; }
```

```
  public Passenger getPassenger(){ return passenger; }
  public BigDecimal getFare()   { return fare; }

  @Override public String toString() {
    return "Seat %s | %s | Fare ₹%s".formatted(seatNo, passenger.getName(), fare);
  }

  @Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Ticket t)) return false;
    return Objects.equals(seatNo, t.seatNo) &&
Objects.equals(passenger.getName(), t.passenger.getName());
  }

  @Override public int hashCode() {
    return Objects.hash(seatNo, passenger.getName());
  }
}

class TicketDemo {
  public static void main(String[] args) {
    Passenger p = new Passenger("Rajat", "rajat@example.com");
    Ticket t = new Ticket("12A", p, new BigDecimal("799.00"));
    System.out.println(t);
  }
}
```

Reasoning to interviewer:

- Composition models reality: a Ticket has-a Passenger.
- BigDecimal prevents fare rounding issues; seatNo as String supports various seat schemas.
- equals/hashCode can be adapted (e.g., to use PNR/bookingId if available).

Future scope:

- Add TicketStatus (BOOKED/CANCELLED/REFUNDED), travel time, and PNR.
- Enforce seat allocation rules in a SeatMap service; apply concurrency control when multiple users book the same seat.

Constructors (Default, Parameterized, Copy, Chaining, Private)

Scenario 6: Default constructor, Courier assigns a tracking ID.

A courier assigns a default tracking ID if none is provided. How will you use a default constructor?

Answer:

In courier systems, every parcel needs a unique tracking ID. If the user doesn't provide one, the system should auto-generate it. This is where a default constructor comes in handy.

- I'll use UUID.randomUUID() to generate unique IDs.
- If recipient details are missing, I'll default them to "Unknown".
- This ensures that even without explicit data, objects are always valid.

Code:

```
import java.util.UUID;

class Shipment {
   private final String trackingId;
   private final String recipient;

   // Default constructor assigns tracking ID
   public Shipment() {
      this.trackingId = UUID.randomUUID().toString();
      this.recipient = "Unknown";
   }

   // Parameterized constructor if details are provided
   public Shipment(String recipient, String trackingId) {
      this.recipient = (recipient == null || recipient.isBlank()) ? "Unknown" :
recipient.trim();
      this.trackingId = (trackingId == null || trackingId.isBlank())
            ? UUID.randomUUID().toString()
            : trackingId.trim();
   }

   @Override
   public String toString() {
      return "Shipment [Recipient=" + recipient + ", TrackingId=" + trackingId + "]";
   }

   public static void main(String[] args) {
      Shipment s1 = new Shipment();
      Shipment s2 = new Shipment("Rajat", null);
      System.out.println(s1);
```

```
        System.out.println(s2);
    }
}
```

Reasoning to interviewer:

- A default constructor guarantees every object starts in a valid state, even with no data passed.
- Using UUID ensures uniqueness across millions of shipments.
- This avoids errors like “null tracking ID” later in the system.

Future scope:

- Add prefixes for tracking IDs based on region (e.g., IN-2025-XXXX).
- Integrate with an ID generation service (like Twitter Snowflake) for distributed scalability.

Scenario 7: Parameterized constructor, Merchandise(title, unit price)

A retail store wants to initialize Merchandise objects with title and unit price at creation. How will you use a parameterized constructor?

Answer:

In retail, merchandise must always have a title and a unit price. This makes a parameterized constructor ideal.

- title → String (trimmed and validated).
- price → BigDecimal to avoid floating-point errors.
- Validate: price ≥ 0, title not empty.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

class Merchandise {
  private final String title;
  private final BigDecimal unitPrice;

  public Merchandise(String title, BigDecimal unitPrice) {
    if (title == null || title.isBlank())
      throw new IllegalArgumentException("Title is required");
    if (unitPrice == null || unitPrice.signum() < 0)
      throw new IllegalArgumentException("Price must be >= 0");

    this.title = title.trim();
    this.unitPrice = unitPrice.setScale(2, RoundingMode.HALF_UP);
  }

  @Override
  public String toString() {
    return title + " | Price: ₹" + unitPrice;
  }

  public static void main(String[] args) {
    Merchandise m = new Merchandise("Notebook", new BigDecimal("45.75"));
    System.out.println(m);
  }
}
```

Reasoning to interviewer:

- Parameterized constructors enforce mandatory fields at creation.
- Using BigDecimal prevents precision issues common with double.

- Encapsulation ensures no invalid objects exist (like a product with negative price).

Future scope:

- Add currency support (INR, USD).
- Use a builder pattern for optional fields like SKU, category, discount.

Scenario 8: Copy constructor, Duplicate Ticket for another passenger

A travel app allows duplicating a Ticket object to quickly book for another passenger. How can you implement a copy constructor?

Answer:

In travel apps, sometimes you want to duplicate a ticket (same fare, route, timing) but assign a new passenger.

- A copy constructor lets us clone an object.
- For immutables like fare, reuse directly. For mutable nested objects, decide between shallow copy or deep copy.
- Here, I'll deep copy passenger details.

Code:

```
import java.math.BigDecimal;

class Passenger {
   private final String name;
   public Passenger(String name) { this.name = name; }
   public String getName() { return name; }
}

class Ticket {
   private final String seatNo;
   private final Passenger passenger;
   private final BigDecimal fare;

   // Regular constructor
   public Ticket(String seatNo, Passenger passenger, BigDecimal fare) {
      this.seatNo = seatNo;
      this.passenger = passenger;
      this.fare = fare;
   }

   // Copy constructor
   public Ticket(Ticket other, Passenger newPassenger) {
      this.seatNo = other.seatNo;
      this.fare = other.fare;
      this.passenger = newPassenger;
   }

   @Override
   public String toString() {
      return "Seat: " + seatNo + ", Passenger: " + passenger.getName() + ", Fare: ₹" +
fare;
   }
```

```
  public static void main(String[] args) {
    Ticket t1 = new Ticket("12A", new Passenger("Rajat"), new BigDecimal("799"));
    Ticket t2 = new Ticket(t1, new Passenger("Anita")); // duplicate for new passenger
    System.out.println(t1);
    System.out.println(t2);
  }
}
```

Reasoning to interviewer:

- Copy constructors are clean for duplicating objects with modifications.
- This is safer than cloning (clone()) because we control what's deep copied.
- In travel, you often reuse booking details but switch passenger → this maps perfectly.

Future scope:

- Add unique booking IDs (PNR).
- Integrate with seat assignment rules (e.g., auto-assign next seat if original is taken).

Scenario 9: Constructor chaining, Car (model only, or model+price)

In a vehicle system, Car has multiple constructors. One accepts only model name, another accepts model name + price. How will you implement constructor chaining?

Answer:

When a class has multiple constructors, repeating validation leads to duplication. Instead, we use constructor chaining with this().

- One constructor delegates to another.
- Ensures **single point of validation**.

Code:

```
import java.math.BigDecimal;

class Car {
   private final String model;
   private final BigDecimal price;

   // Constructor with only model
   public Car(String model) {
      this(model, BigDecimal.ZERO); // chaining
   }

   // Constructor with model + price
   public Car(String model, BigDecimal price) {
      if (model == null || model.isBlank())
         throw new IllegalArgumentException("Model required");
      if (price == null || price.signum() < 0)
         throw new IllegalArgumentException("Price must be >= 0");
      this.model = model.trim();
      this.price = price;
   }

   @Override
   public String toString() {
      return model + " | ₹" + price;
   }

   public static void main(String[] args) {
      Car c1 = new Car("Honda City");
      Car c2 = new Car("BMW", new BigDecimal("4500000"));
      System.out.println(c1);
      System.out.println(c2);
   }
}
```

Reasoning to interviewer:

- this(model, BigDecimal.ZERO) shows constructor chaining.
- Prevents duplicate validation logic → DRY principle.
- Default price = 0 allows partial data without breaking invariants.

Future scope:

- Add builder for optional attributes (color, engine type, year).
- Pull price from a catalog service.

Scenario 10: Private constructor, Restrict instantiation (ConfigManager)

A class must not allow direct object creation (e.g., ConfigManager). How will you restrict instantiation with a private constructor?

Answer:

Sometimes we want only one instance (singleton). Example: ConfigManager that loads system config.

- Use a private constructor to block new.
- Expose a static factory method to return the instance.

Code:

```
class ConfigManager {

  private static ConfigManager instance;

  // Private constructor
  private ConfigManager() {
    System.out.println("Loading configs...");
  }

  // Static factory (lazy init)
  public static synchronized ConfigManager getInstance() {
    if (instance == null) {
      instance = new ConfigManager();
    }
    return instance;
  }

  public void showConfig() {
    System.out.println("Configs ready to use!");
  }

  public static void main(String[] args) {
    ConfigManager c1 = ConfigManager.getInstance();
    ConfigManager c2 = ConfigManager.getInstance();
    System.out.println(c1 == c2); // true (same instance)
  }
}
```

Reasoning to interviewer:

- Private constructor enforces controlled instantiation.
- Lazy initialization ensures configs load only when needed.
- Thread-safety ensured with synchronized.

Future scope:

- Use double-checked locking with volatile for performance.
- Store configs in an external file (JSON/DB) and load at startup.

Inheritance (Single, Multilevel, Hierarchical, Interface-Based)

Scenario 11: Single Inheritance - Transport → Truck, Scooter

A logistics company has a Transport base class and specialized Truck and Scooter classes. How will you use single inheritance?

Answer:

I'll define a base Transport with common fields (registration, capacityKg, baseSpeedKmph) and behaviors like eta(km) and costPerKm(). Truck and Scooter will extend Transport, override eta/costPerKm, and add class-specific fields (e.g., axles for Truck, batteryWh for Scooter). This shows single inheritance: one parent, one direct child type.

Code:

```
abstract class Transport {
   private final String registration;
   private final int capacityKg;
   private final double baseSpeedKmph;

   protected Transport(String registration, int capacityKg, double baseSpeedKmph) {
      if (registration == null || registration.isBlank()) throw new
IllegalArgumentException("Registration required");
      if (capacityKg <= 0 || baseSpeedKmph <= 0) throw new
IllegalArgumentException("Positive values required");
      this.registration = registration.trim();
      this.capacityKg = capacityKg;
      this.baseSpeedKmph = baseSpeedKmph;
   }

   public String getRegistration() { return registration; }
   public int getCapacityKg() { return capacityKg; }
   public double getBaseSpeedKmph() { return baseSpeedKmph; }

   public double etaHours(double distanceKm) { // default ETA = distance / base
speed
      return distanceKm / baseSpeedKmph;
   }
   public abstract double costPerKm();

   @Override public String toString() {
      return "%s | cap:%dkg | speed:%.1fkmph".formatted(registration, capacityKg,
baseSpeedKmph);
   }
}

class Truck extends Transport {
```

```
  private final int axles;

  public Truck(String reg, int capacityKg, double speedKmph, int axles) {
    super(reg, capacityKg, speedKmph);
    if (axles <= 0) throw new IllegalArgumentException("Axles > 0");
    this.axles = axles;
  }

  @Override public double costPerKm() {
    // Heavier vehicles: higher base + axle factor
    return 18.0 + axles * 2.5;
  }

  @Override public double etaHours(double distanceKm) {
    // Trucks slow down under heavy load: add 10% to ETA if > 70% capacity used
(illustrative)
    double eta = super.etaHours(distanceKm);
    return getCapacityKg() > 0 ? eta * 1.10 : eta;
  }

  public int getAxles() { return axles; }
}

class Scooter extends Transport {
  private final int batteryWh;

  public Scooter(String reg, int capacityKg, double speedKmph, int batteryWh) {
    super(reg, capacityKg, speedKmph);
    if (batteryWh <= 0) throw new IllegalArgumentException("batteryWh > 0");
    this.batteryWh = batteryWh;
  }

  @Override public double costPerKm() {
    // Light EV: cheap per km
    return 2.8;
  }

  @Override public double etaHours(double distanceKm) {
    // Scooters take city shortcuts → 5% faster on average (illustrative)
    return super.etaHours(distanceKm) * 0.95;
  }

  public int getBatteryWh() { return batteryWh; }
}

class Demo {
  public static void main(String[] args) {
```

```
    Transport t1 = new Truck("MH12-TRK-001", 12000, 55, 4);
    Transport t2 = new Scooter("MH12-SCT-777", 40, 35, 1200);
    System.out.println(t1 + " | ₹/km: " + t1.costPerKm() + " | ETA(100km): " +
t1.etaHours(100));
    System.out.println(t2 + " | ₹/km: " + t2.costPerKm() + " | ETA(20km): " +
t2.etaHours(20));
  }
}
```

Reasoning to interviewer:

- Common properties/behavior live in Transport; children override specifics → clean single inheritance.
- abstract double costPerKm() forces each vehicle to define its own cost model.
- super.etaHours() shows reuse + specialization.
- Encapsulation via private fields; safe construction via validation.

Future scope:

- Add load(int kg) to check capacity before dispatch.
- Strategy for pricing (peak-hour pricing) instead of hardcoding in subclasses.
- Persist different vehicle types with single table inheritance or joined strategy.

Scenario 12: Multilevel Inheritance - Person → Teacher → Professor

An education system has Person → Teacher → Professor. Professors inherit attributes of Teachers and add research-related behavior. How will you design this?

Answer:

I'll model Person (name, id, contact). Teacher extends Person with subject, teach(). Professor extends Teacher adding researchArea, publishPaper(), and overriding getRole() to reflect seniority. This demonstrates multilevel inheritance and reuse at each layer.

Code:

```
class Person {
   private final String name;
   private final String id;

   public Person(String name, String id) {
      if (name == null || name.isBlank() || id == null || id.isBlank())
         throw new IllegalArgumentException("Name and ID required");
      this.name = name.trim();
      this.id = id.trim();
   }

   public String getName() { return name; }
   public String getId() { return id; }
   public String getRole() { return "Person"; }

   @Override public String toString() { return "%s (%s) - %s".formatted(name, id,
getRole()); }
}

class Teacher extends Person {
   private final String subject;

   public Teacher(String name, String id, String subject) {
      super(name, id);
      if (subject == null || subject.isBlank()) throw new
IllegalArgumentException("Subject required");
      this.subject = subject.trim();
   }

   public String getSubject() { return subject; }
   public void teach() { System.out.println(getName() + " teaches " + subject); }

   @Override public String getRole() { return "Teacher"; }
}
```

```
class Professor extends Teacher {
  private final String researchArea;

  public Professor(String name, String id, String subject, String researchArea) {
    super(name, id, subject);
    if (researchArea == null || researchArea.isBlank()) throw new
IllegalArgumentException("Research area required");
    this.researchArea = researchArea.trim();
  }

  public String getResearchArea() { return researchArea; }
  public void publishPaper(String title) {
    System.out.println(getName() + " published: \"" + title + "\" in " + researchArea);
  }

  @Override public String getRole() { return "Professor"; }
}

class EduDemo {
  public static void main(String[] args) {
    Professor p = new Professor("Dr. Mehta", "FAC-009", "Distributed Systems",
"Consensus Algorithms");
    System.out.println(p);
    p.teach();
    p.publishPaper("A Fast Raft Variant for Edge Clusters");
  }
}
```

Reasoning to interviewer:

- Each level adds responsibilities; Professor inherits all Teacher behavior and extends it.
- Overriding getRole() demonstrates runtime polymorphism across the chain.
- Constructors use super(...) to ensure base state is initialized first.

Future scope:

- Add Staff mixins via interfaces (e.g., AdvisoryBoardMember).
- Role-based permissions in a campus system using the inheritance hierarchy.
- Persist hierarchy with ORM; map different tables per class if needed.

Scenario 13: Hierarchical Inheritance - MedProfessional → Cardiologist, Orthopedist

A hospital has multiple types of doctors (Cardiologist, Orthopedist) inheriting from a common MedProfessional. How can hierarchical inheritance be used?

Answer:

MedProfessional will define shared fields (name, licenseNo) and an abstract treat(Patient) method. Cardiologist and Orthopedist both extend MedProfessional and implement treat with domain-specific logic. This shows one parent with many children.

Code:

```
class Patient {
   private final String name;
   private final String complaint;
   public Patient(String name, String complaint) {
      if (name == null || name.isBlank() || complaint == null || complaint.isBlank())
         throw new IllegalArgumentException("Name & complaint required");
      this.name = name.trim();
      this.complaint = complaint.trim();
   }
   public String getName() { return name; }
   public String getComplaint() { return complaint; }
}

abstract class MedProfessional {
   private final String name;
   private final String licenseNo;

   protected MedProfessional(String name, String licenseNo) {
      if (name == null || name.isBlank() || licenseNo == null || licenseNo.isBlank())
         throw new IllegalArgumentException("Name & license required");
      this.name = name.trim();
      this.licenseNo = licenseNo.trim();
   }

   public String getName() { return name; }
   public String getLicenseNo() { return licenseNo; }

   public abstract void treat(Patient p);

   @Override public String toString() { return "%s (Lic:%s)".formatted(name,
licenseNo); }
}

class Cardiologist extends MedProfessional {
   public Cardiologist(String name, String licenseNo) { super(name, licenseNo); }
```

```
  @Override public void treat(Patient p) {
    System.out.println(getName() + " treating " + p.getName() + " for cardiac issue: " +
p.getComplaint());
    // e.g., order ECG, prescribe beta blockers (domain-illustrative)
  }
}

class Orthopedist extends MedProfessional {
  public Orthopedist(String name, String licenseNo) { super(name, licenseNo); }

  @Override public void treat(Patient p) {
    System.out.println(getName() + " treating " + p.getName() + " for ortho issue: " +
p.getComplaint());
    // e.g., X-ray, cast, physiotherapy plan (domain-illustrative)
  }
}

class HospitalDemo {
  public static void main(String[] args) {
    Patient a = new Patient("Riya", "Shortness of breath");
    Patient b = new Patient("Karan", "Knee pain after fall");
    MedProfessional doc1 = new Cardiologist("Dr. Rao", "MH-MED-1102");
    MedProfessional doc2 = new Orthopedist("Dr. Shah", "MH-MED-2210");
    doc1.treat(a);
    doc2.treat(b);
  }
}
```

Reasoning to interviewer:

- Hierarchical inheritance models “shared base + specialized children.”
- Abstract treat(Patient) enforces a contract; subclasses provide concrete behavior.
- Using base type MedProfessional in variables demonstrates polymorphism (same call, different behavior).

Future scope:

- Add more specializations (Neurologist, Dermatologist).
- Introduce common interfaces (Surgeon, TeleConsult) orthogonal to specialization.
- Scheduling and triage strategies can accept MedProfessional and route by type.

Scenario 14: Inheritance to extend Employee → Manager (no code duplication)

An organization wants managers to have extra capabilities on top of employees, without duplicating code. How would inheritance help?

Answer:

Use single inheritance: keep shared state/behavior in Employee (id, name, salary, attendance) and extend with Manager for extra capabilities (approve leave, view reports, assign tasks). Override role-specific behavior (e.g., getRole()), and reuse common methods via super.

Code:

```
class Employee {
   private final String id;
   private final String name;
   private double salary;

   public Employee(String id, String name, double salary) {
      if (id == null || id.isBlank() || name == null || name.isBlank() || salary < 0)
         throw new IllegalArgumentException("Invalid employee data");
      this.id = id.trim(); this.name = name.trim(); this.salary = salary;
   }

   public String getId() { return id; }
   public String getName() { return name; }
   public double getSalary() { return salary; }
   public void giveRaise(double percent) { salary += salary * (percent / 100.0); }
   public String getRole() { return "Employee"; }

   @Override public String toString() { return "%s %s (₹%.2f)".formatted(getRole(),
name, salary); }
}

class Manager extends Employee {
   public Manager(String id, String name, double salary) { super(id, name, salary); }

   public boolean approveLeave(String employeeId, int days) {
      if (days <= 0) throw new IllegalArgumentException("Days must be > 0");
      // simple rule: approve up to 10 days
      return days <= 10;
   }
   public void assignTask(String employeeId, String task) {
      System.out.println("Assigned '" + task + "' to " + employeeId);
   }
   @Override public String getRole() { return "Manager"; }
```

```
  public static void main(String[] args) {
    Manager m = new Manager("E101","Anita", 900000);
    m.giveRaise(5); // reused from Employee
    System.out.println(m + " | approve 5 days? " + m.approveLeave("E102",5));
  }
}
```

Reasoning to interviewer:

- Shared fields/behaviors live in Employee; Manager adds capabilities without copy-paste.
- Overriding getRole() demonstrates polymorphism when printing or authorizing.
- If policies change, update once in Employee (raise logic, validations) - managers inherit it.

Future scope:

- Add Lead or Director as further specializations; extract approval policy to a LeavePolicy strategy to avoid hardcoding.

Scenario 15: Multilevel banking - Account → SavingsAccount → HighInterestSavingsAccount

In banking: Account → SavingsAccount → HighInterestSavingsAccount. Each level adds new rules. How do you represent this hierarchy?

Answer:

Model a base Account with core money operations; SavingsAccount adds minimum balance and base interest; HighInterestSavingsAccount tightens rules (higher min balance, withdrawal limits, bonus interest conditions). Use overriding for business rules, and super to reuse safe money mechanics.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

class Account {
   protected BigDecimal balance = BigDecimal.ZERO.setScale(2,
RoundingMode.HALF_UP);

   public Account(BigDecimal opening) {
      deposit(opening == null ? BigDecimal.ZERO : opening);
   }
   public void deposit(BigDecimal amt) {
      if (amt == null || amt.signum() < 0) throw new IllegalArgumentException("Invalid
deposit");
      balance = balance.add(amt).setScale(2, RoundingMode.HALF_UP);
   }
   public void withdraw(BigDecimal amt) {
      if (amt == null || amt.signum() <= 0) throw new IllegalArgumentException("Invalid
withdrawal");
      if (balance.compareTo(amt) < 0) throw new IllegalStateException("Insufficient
funds");
      balance = balance.subtract(amt).setScale(2, RoundingMode.HALF_UP);
   }
   public BigDecimal getBalance() { return balance; }
   public String getType() { return "Account"; }
}

class SavingsAccount extends Account {
   protected final BigDecimal minBalance;
   protected final BigDecimal annualRate; // e.g., 0.035 = 3.5%

   public SavingsAccount(BigDecimal opening, BigDecimal minBalance, BigDecimal
annualRate) {
      super(opening);
```

```
    this.minBalance = minBalance; this.annualRate = annualRate;
    if (balance.compareTo(minBalance) < 0) throw new
IllegalStateException("Opening < min balance");
  }

  @Override public void withdraw(BigDecimal amt) {
    super.withdraw(amt);
    if (balance.compareTo(minBalance) < 0) {
      // revert and fail
      deposit(amt);
      throw new IllegalStateException("Cannot go below minimum balance");
    }
  }

  public BigDecimal monthlyInterest() {
    BigDecimal monthly = balance.multiply(annualRate).divide(new
BigDecimal("12"), 6, RoundingMode.HALF_UP);
    return monthly.setScale(2, RoundingMode.HALF_UP);
  }

  @Override public String getType() { return "SavingsAccount"; }
}

class HighInterestSavingsAccount extends SavingsAccount {
  private int withdrawalsThisMonth = 0;
  private final int withdrawalLimit;

  public HighInterestSavingsAccount(BigDecimal opening) {
    super(opening, new BigDecimal("20000.00"), new BigDecimal("0.055")); // stricter
min + higher rate
    this.withdrawalLimit = 3;
  }

  @Override public void withdraw(BigDecimal amt) {
    if (withdrawalsThisMonth >= withdrawalLimit)
      throw new IllegalStateException("Monthly withdrawal limit reached");
    super.withdraw(amt);
    withdrawalsThisMonth++;
  }

  // Example: bonus interest if balance >= 1 lakh
  public BigDecimal monthlyInterest() {
    BigDecimal base = super.monthlyInterest();
    if (balance.compareTo(new BigDecimal("100000.00")) >= 0) {
      base = base.add(base.multiply(new BigDecimal("0.10"))); // +10% bonus on
interest
    }
```

```
    return base.setScale(2, RoundingMode.HALF_UP);
  }

  @Override public String getType() { return "HighInterestSavingsAccount"; }

  public static void main(String[] args) {
    HighInterestSavingsAccount acc = new HighInterestSavingsAccount(new
BigDecimal("120000"));
    System.out.println(acc.getType() + " | Bal: " + acc.getBalance() + " | MI: " +
acc.monthlyInterest());
    acc.withdraw(new BigDecimal("5000"));
    System.out.println("After withdraw: " + acc.getBalance());
  }
}
```

Reasoning to interviewer:

- Core invariants (no negative, scale) live in Account.
- SavingsAccount tightens behavior (minimum balance) via override.
- HighInterestSavingsAccount layers more rules (limits, bonus) without touching lower classes - clean multilevel extension.

Future scope:

- Extract interest computation to a RatePolicy strategy; persist monthly counters; add fees when violating rules.

Scenario 16: Multiple behavior via interfaces - SmartPhone as Camera + MusicPlayer

A SmartPhone must behave both like a Camera and a MusicPlayer. Since Java doesn't support multiple inheritance with classes, how do you achieve this with interfaces?

Answer:

Since Java doesn't allow multiple inheritance with classes, define Camera and MusicPlayer as interfaces. SmartPhone implements both. If default methods conflict, override in SmartPhone to resolve ambiguity.

Code:

```
interface Camera {
   void click();
   default void open() { System.out.println("Camera app opened"); }
}
interface MusicPlayer {
   void play(String track);
   default void open() { System.out.println("Music player opened"); }
}

class SmartPhone implements Camera, MusicPlayer {
   @Override public void click() { System.out.println("Photo captured in HDR"); }
   @Override public void play(String track) { System.out.println("Playing: " + track); }

   // Resolve default method conflict explicitly
   @Override public void open() {
      Camera.super.open(); // or provide unified UI
      MusicPlayer.super.open();
      System.out.println("Unified launcher ready");
   }

   public static void main(String[] args) {
      SmartPhone sp = new SmartPhone();
      sp.open();         // resolves both opens
      sp.click();        // camera behavior
      sp.play("A.R. Rahman - Melody");
   }
}
```

Reasoning to interviewer:

- Interfaces give multiple inheritance of type without state issues.
- Default method conflict is intentionally resolved in the implementing class, avoids the diamond problem.
- Keeps concerns separate and testable (mock Camera/MusicPlayer in unit tests).

Future scope:

- Add more roles (GPS, NFCDevice) as interfaces.
- Compose behavior with DI (swap MusicPlayer implementation for streaming vs local).

Method Overloading

Scenario 17: MathTool with overloaded add methods

A developer creates a MathTool that can add 2, 3, or 4 integers. How will you design overloaded methods?

Answer:

We need a MathTool that can add different counts of integers. Instead of writing differently named methods (addTwo, addThree...), we can use method overloading: same method name add(), but with different parameter lists.

- Overloading is decided at compile time (static polymorphism).
- I'll provide add(int a, int b), add(int a, int b, int c), add(int a, int b, int c, int d).
- This keeps the API consistent (MathTool.add(...)).

Code:

```
class MathTool {
  public int add(int a, int b) {
    return a + b;
  }

  public int add(int a, int b, int c) {
    return a + b + c;
  }

  public int add(int a, int b, int c, int d) {
    return a + b + c + d;
  }

  public static void main(String[] args) {
    MathTool tool = new MathTool();
    System.out.println("2+3 = " + tool.add(2,3));
    System.out.println("2+3+4 = " + tool.add(2,3,4));
    System.out.println("2+3+4+5 = " + tool.add(2,3,4,5));
  }
}
```

Reasoning to interviewer:

- Overloading keeps method naming consistent, avoiding clutter.
- The compiler chooses the right method based on argument count.
- Each method is optimized for its case, but I could also use varargs if I wanted arbitrary numbers.

Future scope:

- Add add(double a, double b) for floating-point addition.
- Provide a add(int... nums) varargs method to handle any number of integers.

Scenario 18: E-store discounts for single vs multiple products

An e-store applies different discounts depending on whether 1 product or multiple products are purchased. How can you use method overloading?

Answer:

In e-commerce, discounts vary:

- If a single product is purchased → apply a flat percentage discount.
- If multiple products are purchased → calculate combined price and apply a bulk discount.
  Here, method overloading can provide two applyDiscount methods: one takes a double price, another takes an array of prices.

Code:

```
class DiscountCalculator {
   // Single product discount
   public double applyDiscount(double price) {
      double discounted = price * 0.9; // 10% off
      return Math.round(discounted * 100.0) / 100.0;
   }

   // Multiple products discount
   public double applyDiscount(double[] prices) {
      double total = 0;
      for (double p : prices) total += p;
      double discounted = total * 0.85; // 15% off for bulk
      return Math.round(discounted * 100.0) / 100.0;
   }

   public static void main(String[] args) {
      DiscountCalculator calc = new DiscountCalculator();
      System.out.println("Single: " + calc.applyDiscount(1000));
      System.out.println("Bulk: " + calc.applyDiscount(new double[]{500, 600, 700}));
   }
}
```

sReasoning to interviewer:

- Overloaded methods let me handle different scenarios (single vs multiple products) with the same method name.
- Cleaner API: applyDiscount() works in both contexts, avoiding method name explosion like applySingleDiscount vs applyBulkDiscount.
- Easy for clients to remember → reduces API confusion.

Future scope:

- Add overloads for applyDiscount(Product p) or applyDiscount(List<Product>).
- Extend with coupon codes, seasonal sales, loyalty points.

Scenario 19: Banking transfers via account number vs phone number

A banking app allows transferring money: one method transfers to an account number, another to a phone number. How can method overloading handle this?

Answer:

In banking, money transfers can go to:

- Another account using account number + IFSC.
- A mobile wallet using phone number.
  I can overload transfer methods: one accepts (String accountNumber, String ifsc, double amount), another (String phoneNumber, double amount).

Code:

```
class BankApp {
   // Transfer via account number
   public void transfer(String accountNumber, String ifsc, double amount) {
      System.out.println("Transferred ₹" + amount + " to account " + accountNumber +
" (IFSC: " + ifsc + ")");
   }

   // Transfer via phone number
   public void transfer(String phoneNumber, double amount) {
      System.out.println("Transferred ₹" + amount + " to mobile number " +
phoneNumber);
   }

   public static void main(String[] args) {
      BankApp app = new BankApp();
      app.transfer("1234567890", "HDFC0001234", 5000);   // Account transfer
      app.transfer("9876543210", 1500);              // Phone transfer
   }
}
```

Reasoning to interviewer:

- Same logical action (transfer) → different parameter sets.
- Overloading prevents duplicate method names like transferToAccount() vs transferToPhone().
- Type/parameter list ensures the compiler selects the correct method.

Future scope:

- Add overloads for transfer(Account target, double amount).
- Add optional remark or currencyCode parameter using method overloading or builders.
- Integrate validations: phone must be 10 digits, account + IFSC format validation.

Scenario 20: Overloaded sendMessage() - text, image, video in a messaging app

In a messaging app, the sendMessage() method can send text, image, or video. How will you overload this method?

Answer:

I'll overload sendMessage() by parameter types to keep a single, memorable API name.

- **Text** → sendMessage(String chatId, String text)
- **Image** → sendMessage(String chatId, byte[] imageBytes, String mimeType)
- **Video** → sendMessage(String chatId, byte[] videoBytes, String mimeType, int durationSec)
  Different signatures avoid ambiguity and let the compiler pick the right method at compile time.

Code:

```
class Messenger {

  // 1) Text
  public void sendMessage(String chatId, String text) {
    if (chatId == null || chatId.isBlank()) throw new IllegalArgumentException("chatId
required");
    if (text == null || text.isBlank()) throw new IllegalArgumentException("text
required");
    // persist and dispatch text
    System.out.println("Text to " + chatId + ": " + text);
  }

  // 2) Image (PNG/JPEG, etc.)
  public void sendMessage(String chatId, byte[] imageBytes, String mimeType) {
    if (chatId == null || chatId.isBlank()) throw new IllegalArgumentException("chatId
required");
    if (imageBytes == null || imageBytes.length == 0) throw new
IllegalArgumentException("image required");
    if (mimeType == null || !mimeType.startsWith("image/")) throw new
IllegalArgumentException("invalid image mime");
    // store blob, generate CDN URL, dispatch
    System.out.println("Image to " + chatId + " (" + mimeType + ", " +
imageBytes.length + " bytes)");
  }

  // 3) Video (with duration metadata)
```

```
  public void sendMessage(String chatId, byte[] videoBytes, String mimeType, int
durationSec) {
    if (chatId == null || chatId.isBlank()) throw new IllegalArgumentException("chatId
required");
    if (videoBytes == null || videoBytes.length == 0) throw new
IllegalArgumentException("video required");
    if (mimeType == null || !mimeType.startsWith("video/")) throw new
IllegalArgumentException("invalid video mime");
    if (durationSec <= 0) throw new IllegalArgumentException("duration > 0
required");
    // transcode/enqueue thumbnail generation, dispatch
    System.out.println("Video to " + chatId + " (" + mimeType + ", " + durationSec + "s,
" + videoBytes.length + " bytes)");
  }

  public static void main(String[] args) {
    Messenger m = new Messenger();
    m.sendMessage("chat-42", "Hello 👋");
    m.sendMessage("chat-42", new byte[1024], "image/png");
    m.sendMessage("chat-42", new byte[2048], "video/mp4", 15);
  }
}
```

Reasoning to interviewer:

- Same verb “sendMessage” expresses one capability; overloading tailors inputs per media type.
- Using distinct parameter lists (String, byte[]+mime, byte[]+mime+duration) lets the compiler resolve which method to call.
- Validation sits at the boundary (null/empty/mime checks) so invalid payloads never enter the pipeline.

Future scope:

- Add overloads for sending documents, locations, or contact cards.
- Replace raw byte[] with InputStream for large payloads; add async variants (CompletableFuture<Void>).
- Introduce a Message builder for advanced options (caption, mentions, reply-to).

Scenario 21: Overloaded bookTicket() - passenger ID vs passport number in airline reservations

In an airline reservation system, bookTicket() can accept either a passenger ID or passport number. How would you design this?

Answer:

I'll expose two overloads to reflect two identification flows.

- **Domestic/known customer:** bookTicket(String passengerId, String flightNo, String date)
- **International/new customer:** bookTicket(String passportNo, String nationality, String flightNo, String date)
  Different parameters prevent confusion and let the booking logic fetch/verify identity correctly.

Code:

```
class AirlineReservation {

  // 1) Known passenger (already in system)
  public String bookTicket(String passengerId, String flightNo, String dateIso) {
    validateFlight(flightNo, dateIso);
    if (passengerId == null || passengerId.isBlank()) throw new
IllegalArgumentException("passengerId required");
    // lookup passenger profile, seat availability, price rules...
    String pnr = generatePnr();
    System.out.println("Booked (existing): " + pnr + " for PID " + passengerId + " on " +
flightNo + " @ " + dateIso);
    return pnr;
  }

  // 2) New/International passenger via passport
  public String bookTicket(String passportNo, String nationality, String flightNo, String
dateIso) {
    validateFlight(flightNo, dateIso);
    if (passportNo == null || passportNo.isBlank()) throw new
IllegalArgumentException("passport required");
    if (nationality == null || nationality.isBlank()) throw new
IllegalArgumentException("nationality required");
    // verify passport format, API checks, create provisional passenger profile...
    String pnr = generatePnr();
    System.out.println("Booked (passport): " + pnr + " for " + passportNo + " [" +
nationality + "] on " + flightNo + " @ " + dateIso);
    return pnr;
```

```
  }

  private void validateFlight(String flightNo, String dateIso) {
    if (flightNo == null || flightNo.isBlank()) throw new
IllegalArgumentException("flightNo required");
    if (dateIso == null || dateIso.isBlank()) throw new IllegalArgumentException("date
required");
    // parse/validate ISO date; check flight exists, not departed, seats available, etc.
  }

  private String generatePnr() {
    return "PNR" + System.nanoTime();
  }

  public static void main(String[] args) {
    AirlineReservation ar = new AirlineReservation();
    ar.bookTicket("PID-8821", "AI202", "2025-09-10");
    ar.bookTicket("P1234567", "IN", "AI305", "2025-10-02");
  }
}
```

Reasoning to interviewer:

- Overloading mirrors business flows while keeping a single action name (bookTicket).
- Parameter lists convey required identity context (ID vs passport + nationality).
- Shared validation is factored into a helper to avoid duplication; each overload adds its own checks.

Future scope:

- Add an overload accepting a Passenger object for richer data (DOB, gender, contact).
- Introduce optional parameters (seat class, meal, baggage) via another overload or a BookingRequest builder.
- Hook in validation services (passport MRZ parsing, no-fly list checks) behind the method boundary.

Method Overriding

Scenario 22: Payroll - Override calculateSalary() for Full-time vs Part-time

In a payroll system, full-time and part-time employees calculate salaries differently. How can you use overriding?

Answer:

Use an abstract Employee with an abstract calculateSalary(). FullTimeEmployee overrides it to use fixed monthly base + allowances; PartTimeEmployee overrides it to use hourlyRate * hoursWorked. Consume them via Employee references to demonstrate runtime polymorphism.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

abstract class Employee {
   private final String id;
   private final String name;

   protected Employee(String id, String name) {
      if (id == null || id.isBlank() || name == null || name.isBlank())
         throw new IllegalArgumentException("Invalid employee");
      this.id = id.trim(); this.name = name.trim();
   }
   public String getId() { return id; }
   public String getName() { return name; }

   public abstract BigDecimal calculateSalary(); // overridden per type
   @Override public String toString() { return name + " (" + id + ")"; }
}

class FullTimeEmployee extends Employee {
   private final BigDecimal baseMonthly;
   private final BigDecimal hra;   // house rent allowance
   private final BigDecimal bonus; // monthly variable

   public FullTimeEmployee(String id, String name,
                           BigDecimal baseMonthly, BigDecimal hra, BigDecimal bonus) {
      super(id, name);
      this.baseMonthly = baseMonthly; this.hra = hra; this.bonus = bonus;
   }
```

```
  @Override public BigDecimal calculateSalary() {
    return baseMonthly.add(hra).add(bonus).setScale(2, RoundingMode.HALF_UP);
  }
}

class PartTimeEmployee extends Employee {
  private final BigDecimal hourlyRate;
  private final int hoursWorked;

  public PartTimeEmployee(String id, String name, BigDecimal hourlyRate, int
hoursWorked) {
    super(id, name);
    this.hourlyRate = hourlyRate; this.hoursWorked = hoursWorked;
  }
  @Override public BigDecimal calculateSalary() {
    return hourlyRate.multiply(BigDecimal.valueOf(hoursWorked))
                .setScale(2, RoundingMode.HALF_UP);
  }
}

class PayrollDemo {
  public static void main(String[] args) {
    List<Employee> staff = List.of(
      new FullTimeEmployee("E101","Anita",
        new BigDecimal("50000"), new BigDecimal("10000"), new
BigDecimal("5000")),
      new PartTimeEmployee("E202","Ravi",
        new BigDecimal("450"), 80)
    );
    staff.forEach(e -> System.out.println(e + " => ₹" + e.calculateSalary()));
  }
}
```

Reasoning to interviewer:

- Overriding isolates salary rules per type while keeping a single API (calculateSalary) for consumers.
- Polymorphism lets payroll iterate List<Employee> without if/else on types.
- BigDecimal prevents currency precision bugs.

Future scope:

- Add ContractorEmployee (daily-rate) without touching existing code.
- Introduce a PayPolicy strategy to swap rules per location/grade.
- Add pro-rating for mid-month joins using overridden hooks.

Scenario 23: University - Override showEnrollmentInfo() for Regular vs Flexible

A university has RegularEnrollment and FlexibleEnrollment classes. Both override a showEnrollmentInfo() method differently.

Answer:

Create an abstract Enrollment with common fields (studentId, program). RegularEnrollment shows fixed credits and semester schedule; FlexibleEnrollment shows selected modules and pay-per-credit. Both override showEnrollmentInfo() differently.

Code:

```
abstract class Enrollment {
   protected final String studentId;
   protected final String program;

   protected Enrollment(String studentId, String program) {
      if (studentId == null || studentId.isBlank() || program == null || program.isBlank())
         throw new IllegalArgumentException("Invalid enrollment");
      this.studentId = studentId.trim(); this.program = program.trim();
   }
   public abstract void showEnrollmentInfo();
}

class RegularEnrollment extends Enrollment {
   private final int fixedCredits;     // e.g., 20 per semester
   private final String semesterWindow; // e.g., "Aug–Dec"

   public RegularEnrollment(String studentId, String program, int fixedCredits, String
window) {
      super(studentId, program);
      this.fixedCredits = fixedCredits; this.semesterWindow = window;
   }
   @Override public void showEnrollmentInfo() {
      System.out.println("[Regular] " + studentId + " - " + program +
         " | Credits: " + fixedCredits + " | Term: " + semesterWindow);
   }
}

class FlexibleEnrollment extends Enrollment {
   private final int chosenCredits;     // variable per term
   private final boolean payPerCredit;
```

```
  public FlexibleEnrollment(String studentId, String program, int chosenCredits,
boolean payPerCredit) {
    super(studentId, program);
    this.chosenCredits = chosenCredits; this.payPerCredit = payPerCredit;
  }
  @Override public void showEnrollmentInfo() {
    System.out.println("[Flexible] " + studentId + " - " + program +
      " | Credits: " + chosenCredits + " | Billing: " + (payPerCredit ? "Per-credit" :
"Flat"));
  }
}

class UniDemo {
  public static void main(String[] args) {
    Enrollment e1 = new RegularEnrollment("S001","B.Tech CSE", 20, "Aug–Dec");
    Enrollment e2 = new FlexibleEnrollment("S002","M.Sc Data", 12, true);
    e1.showEnrollmentInfo();
    e2.showEnrollmentInfo();
  }
}
```

Reasoning to interviewer:

- Shared identity and program fields live in the base; display logic varies by override.
- Consumers rely on the Enrollment abstraction-open for extension, closed for modification.
- Straightforward to add ExecutiveEnrollment later without changing callers.

Future scope:

- Add tuitionFee() as abstract; compute differently per type.
- Attach rules for add/drop deadlines inside each subclass.
- Persist enrollments polymorphically using an ORM.

Scenario 24: Modules with common interface - FinanceModule vs LogisticsModule compute costs

A company has FinanceModule and LogisticsModule, each computing costs differently but following the same interface. How can polymorphism + overriding help?

Answer:

Define an interface CostComputable with computeCost(Order order). FinanceModule and LogisticsModule implement it with different rules (tax/overhead vs distance/fuel). Use the interface type in consuming code to leverage polymorphism.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

class Order {
   final BigDecimal baseValue;  // goods value
   final double distanceKm;     // for logistics
   final double weightKg;

   Order(BigDecimal baseValue, double distanceKm, double weightKg) {
      this.baseValue = baseValue; this.distanceKm = distanceKm; this.weightKg =
weightKg;
   }
}

interface CostComputable {
   BigDecimal computeCost(Order order);
   default String name() { return getClass().getSimpleName(); }
}

class FinanceModule implements CostComputable {
   @Override public BigDecimal computeCost(Order o) {
      BigDecimal tax = o.baseValue.multiply(new BigDecimal("0.18"));     // 18% GST
      BigDecimal overhead = o.baseValue.multiply(new BigDecimal("0.02")); // 2%
finance overhead
      return o.baseValue.add(tax).add(overhead).setScale(2,
RoundingMode.HALF_UP);
   }
}

class LogisticsModule implements CostComputable {
   @Override public BigDecimal computeCost(Order o) {
      BigDecimal perKm = new BigDecimal("9.50"); // base rate
```

```
    BigDecimal fuelSurcharge = new BigDecimal("0.07"); // 7% surcharge
    BigDecimal haul = perKm.multiply(BigDecimal.valueOf(o.distanceKm))
                .add(BigDecimal.valueOf(o.weightKg).multiply(new
BigDecimal("2.0")));
    return haul.add(haul.multiply(fuelSurcharge)).setScale(2,
RoundingMode.HALF_UP);
  }
}

class ModuleDemo {
  public static void main(String[] args) {
    Order order = new Order(new BigDecimal("10000"), 250, 120);
    List<CostComputable> modules = List.of(new FinanceModule(), new
LogisticsModule());
    modules.forEach(m ->
      System.out.println(m.name() + " => ₹" + m.computeCost(order))
    );
  }
}
```

Reasoning to interviewer:

- Interface defines the contract; each module overrides with its own cost logic.
- Polymorphism lets a pipeline treat all modules uniformly (List<CostComputable>), while behavior differs at runtime.
- Extensible: add InsuranceModule later, no change to consumers.

Future scope:

- Chain modules with a CostPipeline that aggregates or prioritizes costs.
- Externalize rates to config/DB; inject via DI for testability.
- Add currency handling and localization for amounts.

Scenario 25: HR, Override calculateBenefits() for Contract vs Permanent employees

In an HR system, both ContractEmployee and PermanentEmployee override the calculateBenefits() method differently. How is overriding applied?

Answer:

Model a common abstraction EmployeeBase with shared identity and an abstract calculateBenefits() that returns the monthly benefits value.

- PermanentEmployee includes medical, PF/employer EPF, gratuity components.
- ContractEmployee has a leaner package (e.g., only insurance stipend or none). Consumers operate on EmployeeBase and rely on runtime polymorphism.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

abstract class EmployeeBase {
   private final String id;
   private final String name;
   protected final BigDecimal baseMonthly;

   protected EmployeeBase(String id, String name, BigDecimal baseMonthly) {
      if (id == null || id.isBlank() || name == null || name.isBlank() || baseMonthly == null
|| baseMonthly.signum() < 0)
         throw new IllegalArgumentException("Invalid employee data");
      this.id = id.trim(); this.name = name.trim(); this.baseMonthly =
baseMonthly.setScale(2, RoundingMode.HALF_UP);
   }

   public String getId() { return id; }
   public String getName() { return name; }

   public abstract BigDecimal calculateBenefits();  // overridden

   @Override public String toString() { return name + " [" + id + "]"; }
}

class PermanentEmployee extends EmployeeBase {
   public PermanentEmployee(String id, String name, BigDecimal baseMonthly) {
      super(id, name, baseMonthly);
   }
   @Override public BigDecimal calculateBenefits() {
```

```
    // Example: HRA 20% + Employer PF 12% + Medical 1500
    BigDecimal hra = baseMonthly.multiply(new BigDecimal("0.20"));
    BigDecimal pf  = baseMonthly.multiply(new BigDecimal("0.12"));
    BigDecimal medical = new BigDecimal("1500.00");
    return hra.add(pf).add(medical).setScale(2, RoundingMode.HALF_UP);
  }
}

class ContractEmployee extends EmployeeBase {
  public ContractEmployee(String id, String name, BigDecimal baseMonthly) {
    super(id, name, baseMonthly);
  }
  @Override public BigDecimal calculateBenefits() {
    // Example: only fixed insurance stipend
    return new BigDecimal("600.00");
  }
}

class HRDemo {
  public static void main(String[] args) {
    List<EmployeeBase> team = List.of(
      new PermanentEmployee("E101","Anita", new BigDecimal("65000")),
      new ContractEmployee("C204","Ravi", new BigDecimal("40000"))
    );
    team.forEach(e -> System.out.println(e + " | Monthly benefits: ₹" +
e.calculateBenefits()));
  }
}
```

Reasoning to interviewer:

- Overriding isolates policy differences while keeping a common API (calculateBenefits()).
- Consumers don't branch on type - they iterate a List<EmployeeBase> and let the JVM dispatch the right method.
- BigDecimal keeps money math precise; components are explicit and easy to tweak.

Future scope:

- Introduce location/grade-specific benefit strategies injected into each subclass.
- Add totalCostToCompany() that combines base + benefits; override for

contractors with markups.

- Persist benefits breakdown for payslip generation and audits.

Scenario 26: Tax system - Override computeTax() for Individual, Company, NGO

A tax calculation system overrides computeTax() differently for individuals, companies, and NGOs. How would you model this?

Answer:

Use an abstract TaxPayer (or an interface) with computeTax(BigDecimal income).

- IndividualTaxPayer applies slab-based tax and rebate/standard deduction.
- CompanyTaxPayer applies a flat corporate rate + surcharge.
- NgoTaxPayer often has exemptions; compute zero or limited tax based on rules.
  Process all uniformly via List<TaxPayer> to demonstrate polymorphism.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

abstract class TaxPayer {
   private final String name;

   protected TaxPayer(String name) {
      if (name == null || name.isBlank()) throw new IllegalArgumentException("Name
required");
      this.name = name.trim();
   }
   public String getName() { return name; }

   public abstract BigDecimal computeTax(BigDecimal annualIncome); // overridden
}

class IndividualTaxPayer extends TaxPayer {
   public IndividualTaxPayer(String name) { super(name); }

   @Override public BigDecimal computeTax(BigDecimal income) {
      if (income == null || income.signum() < 0) throw new
IllegalArgumentException("Income invalid");
      // Illustrative slab (not a real country table):
      // 0–2.5L: 0%, 2.5–5L: 5%, 5–10L: 20%, >10L: 30%; standard deduction 50k
      BigDecimal fiftyK = new BigDecimal("50000");
      BigDecimal taxable = income.subtract(fiftyK).max(BigDecimal.ZERO);
```

```
    BigDecimal tax = BigDecimal.ZERO;
    BigDecimal slabA = new BigDecimal("250000");
    BigDecimal slabB = new BigDecimal("500000");
    BigDecimal slabC = new BigDecimal("1000000");

    if (taxable.compareTo(slabA) > 0) {
      BigDecimal part = taxable.min(slabB).subtract(slabA).max(BigDecimal.ZERO);
      tax = tax.add(part.multiply(new BigDecimal("0.05")));
    }
    if (taxable.compareTo(slabB) > 0) {
      BigDecimal part = taxable.min(slabC).subtract(slabB).max(BigDecimal.ZERO);
      tax = tax.add(part.multiply(new BigDecimal("0.20")));
    }
    if (taxable.compareTo(slabC) > 0) {
      BigDecimal part = taxable.subtract(slabC);
      tax = tax.add(part.multiply(new BigDecimal("0.30")));
    }
    return tax.setScale(2, RoundingMode.HALF_UP);
  }
}

class CompanyTaxPayer extends TaxPayer {
  public CompanyTaxPayer(String name) { super(name); }

  @Override public BigDecimal computeTax(BigDecimal income) {
    if (income == null || income.signum() < 0) throw new
IllegalArgumentException("Income invalid");
    // Example: 22% corporate rate + 5% surcharge on tax
    BigDecimal baseTax = income.multiply(new BigDecimal("0.22"));
    BigDecimal surcharge = baseTax.multiply(new BigDecimal("0.05"));
    return baseTax.add(surcharge).setScale(2, RoundingMode.HALF_UP);
  }
}

class NgoTaxPayer extends TaxPayer {
  public NgoTaxPayer(String name) { super(name); }

  @Override public BigDecimal computeTax(BigDecimal income) {
    if (income == null || income.signum() < 0) throw new
IllegalArgumentException("Income invalid");
    // Example: NGOs exempt unless non-charitable income exceeds threshold
    BigDecimal threshold = new BigDecimal("2000000"); // 20L
    if (income.compareTo(threshold) <= 0) return BigDecimal.ZERO.setScale(2,
RoundingMode.HALF_UP);
    BigDecimal taxablePortion = income.subtract(threshold);
    return taxablePortion.multiply(new BigDecimal("0.10")).setScale(2,
RoundingMode.HALF_UP); // 10% on excess
```

```
  }
}

class TaxDemo {
  public static void main(String[] args) {
    List<TaxPayer> payers = List.of(
      new IndividualTaxPayer("Riya"),
      new CompanyTaxPayer("Acme Pvt Ltd"),
      new NgoTaxPayer("Green Foundation")
    );
    List<BigDecimal> incomes = List.of(
      new BigDecimal("1200000"), // individual
      new BigDecimal("50000000"),// company
      new BigDecimal("1500000")  // NGO
    );
    for (int i = 0; i < payers.size(); i++) {
      System.out.println(payers.get(i).getName() + " => Tax: ₹" +
payers.get(i).computeTax(incomes.get(i)));
    }
  }
}
```

Reasoning to interviewer:

- Overriding captures domain-specific tax rules per entity without if/else cascades.
- Consumers depend on the abstraction (TaxPayer) and stay unchanged as new categories appear, open/closed principle.
- Money math uses BigDecimal with rounding; slab logic split into readable steps.

Future scope:

- Externalize slabs/rates to a configuration or TaxPolicy strategy; inject per financial year.
- Add rebates/credits interfaces (e.g., RebateApplicable) mix-in style.
- Support multiple jurisdictions by composing a chain of policies evaluated per taxpayer.

Access Modifiers

Scenario 27: Digital Library, applying private, default, protected, public

A digital library has attributes that must be private, default, protected, and public depending on usage. How will you apply modifiers?

Answer:

I'll model a LibraryItem where:

- **id** is private (internal identity)
- **shelfCode** is default/package-private (used only within the catalog package)
- **lastBorrowedOn** is protected (needed by subclasses like EBook, AudioBook, possibly in other packages)
- **title** is public (safe to expose)

Code:

```
// file: com/org/library/catalog/LibraryItem.java
package com.org.library.catalog;

import java.time.LocalDate;

public class LibraryItem {
  private final String id;          // private: only this class
      String shelfCode;          // default: only com.org.library.catalog package
  protected LocalDate lastBorrowedOn;   // protected: package + subclasses
  public final String title;         // public: visible to everyone (immutable for safety)

  public LibraryItem(String id, String title) {
    if (id == null || id.isBlank() || title == null || title.isBlank())
      throw new IllegalArgumentException("id/title required");
    this.id = id.trim();
    this.title = title.trim();
  }

  // private field accessed via controlled API
  public String getIdMasked() { return "ITEM-" + id.hashCode(); }

  void assignShelf(String code) { this.shelfCode = code; }            // package use
  protected void markBorrowed(LocalDate date) { this.lastBorrowedOn = date; } //
subclasses
}

// file: com/org/library/media/EBook.java
```

```
package com.org.library.media;

import com.org.library.catalog.LibraryItem;
import java.time.LocalDate;

public class EBook extends LibraryItem {
   public EBook(String id, String title) { super(id, title); }

   public void borrow() {
      // OK: protected member accessible in subclass (even different package)
      markBorrowed(LocalDate.now());
   }
}
```

Reasoning to interviewer:

- Sensitive identity stays private with a safe, public accessor (getIdMasked).
- Operational data shared among catalog helpers (shelfCode) is default-scoped to the package.
- protected supports subclass needs without exposing to all clients.
- public final title is safe to expose and immutable.

Future scope:

- Move protected/default members behind methods if rules grow (auditing, validation).
- Add a LendingPolicy service so only that package can update borrow state.

Scenario 28: Payroll, salary private, name public

In payroll, salary should be private while employee name should be public. How will access modifiers enforce this?

Answer:

I want salary to stay strictly encapsulated; name can be public for display/search. To avoid breaking encapsulation, I'll keep the field private but provide a public getter. If the requirement insists on "public name", I'll make it public final so it can't be reassigned.

Code:

```
package com.org.payroll;

import java.math.BigDecimal;

public class Employee {
  public final String name;        // public & immutable (safe to expose)
  private BigDecimal salary;       // private: strict control

  public Employee(String name, BigDecimal salary) {
    if (name == null || name.isBlank() || salary == null || salary.signum() < 0)
      throw new IllegalArgumentException("Invalid data");
    this.name = name.trim();
    this.salary = salary;
  }

  public BigDecimal getSalary() {    // controlled read
    return salary;
  }
  public void giveRaise(BigDecimal percent) { // controlled write
    if (percent == null) throw new IllegalArgumentException("percent required");
    salary = salary.add(salary.multiply(percent).movePointLeft(2));
  }

  @Override public String toString() {
    return name + " | ₹" + salary;
  }
}
```

Reasoning to interviewer:

- Salary is private to prevent arbitrary writes and support validations/audits.
- Public name is immutable to avoid accidental changes; this balances the requirement with safety.
- Mutations to salary go through business rules (giveRaise) rather than raw setters.

Future scope:

- Add role-based access (only HR can change salary).
- Persist salary changes with an audit trail; expose a read-only projection to other modules.

Scenario 29: Healthcare SSN private, diagnosis protected, name public

In a healthcare app, patient's SSN must be private, diagnosis protected, and general details like name public. How will you design this?

Answer:

In healthcare, PHI needs strict layering:

- ssn is private (never leaked)
- diagnosis is protected (visible to medical subclasses and same package services)
- name is public (generally non-sensitive)

Code:

```
// file: com/health/patient/Patient.java
package com.health.patient;

public class Patient {
  public final String name;     // public for identification in UIs (immutable)
  private final String ssn;     // private PHI: never expose directly
  protected String diagnosis;   // protected: clinicians/subclasses can access

  public Patient(String name, String ssn) {
    if (name == null || name.isBlank() || ssn == null || ssn.isBlank())
      throw new IllegalArgumentException("name/ssn required");
    this.name = name.trim();
    this.ssn = ssn.trim();
  }

  // Only expose masked/hashed versions to external callers
  public String getSsnMasked() {
    int n = Math.max(0, ssn.length() - 4);
    return "*".repeat(n) + ssn.substring(ssn.length() - 4);
  }

  protected void setDiagnosis(String dx) {  // clinical workflows
    if (dx == null || dx.isBlank()) throw new IllegalArgumentException("diagnosis
required");
    this.diagnosis = dx.trim();
  }

  @Override public String toString() {
    return name + " | SSN:" + getSsnMasked() + " | Dx:" + (diagnosis == null ? "-" :
diagnosis);
  }
```

```
}

// file: com/health/clinical/ConsultNote.java
package com.health.clinical;

import com.health.patient.Patient;

public class ConsultNote extends Patient {
  public ConsultNote(String name, String ssn) { super(name, ssn); }

  public void record(String dx) {
    // allowed: protected setter from subclass (even cross-package)
    setDiagnosis(dx);
  }
}
```

Reasoning to interviewer:

- SSN stays private with only a masked accessor so accidental logging/leaks are avoided.
- protected diagnosis supports subclassed clinical workflows and same-package services; it's not exposed to everyone.
- public final name is safe and convenient for UI/search.

Future scope:

- Move diagnosis to a value object with confidentiality levels; enforce access via roles.
- Centralize PHI access through a PrivacyGuard service that logs and authorizes reads.
- Encrypt SSN at rest and restrict decryption to a narrow, audited path.

Scenario 30: E-learning platform, quiz solutions private, scores public

In an e-learning platform, quiz solutions should be private, but scores can be viewed publicly. How will modifiers enforce this?

Answer:

In an online learning system, the solutions to quizzes must never be directly accessible to students (private). On the other hand, scores should be visible to everyone (public).

- I'll declare solutions as private and provide no direct getter for it.
- I'll declare score as public (or public final if immutable), so it can be freely accessed.
- Any logic to reveal solutions (like after submission) will go through a controlled method that decides when/if to share.

Code:

```
class QuizResult {
   private final String[] solutions;   // strictly private
   public final int score;          // safe to expose publicly

   public QuizResult(String[] solutions, int score) {
      if (solutions == null || score < 0) throw new IllegalArgumentException("Invalid
data");
      this.solutions = solutions.clone();
      this.score = score;
   }

   // Controlled access: only admin or teacher can reveal
   public String[] revealSolutions(boolean isTeacher) {
      if (!isTeacher) throw new SecurityException("Unauthorized access to solutions");
      return solutions.clone(); // defensive copy
   }

   @Override
   public String toString() {
      return "Score: " + score; // no solutions exposed
   }

   public static void main(String[] args) {
      QuizResult qr = new QuizResult(new String[]{"A","B","C"}, 8);
      System.out.println(qr); // "Score: 8"
      // qr.solutions is inaccessible
   }
}
```

Reasoning to interviewer:

- solutions is private so no one can directly tamper or leak it.
- score is safe to expose because it doesn't compromise content.
- This enforces encapsulation: sensitive data locked away, public data openly available.

Future scope:

- Add role-based access (teacher vs student vs admin).
- Log every access to solutions for auditing.
- Release solutions only after quiz deadline via time checks.

Scenario 31: HR system, appraisal details visible only inside HR package

In an internal HR system, employee appraisal details must be visible only within the HR package. Which modifier will you use?

Answer:

For internal HR modules, appraisal data should not leak outside. The right modifier here is default (package-private**)** no keyword.

- Default access means only classes in the same package (com.company.hr) can see it.
- External packages (like payroll, finance, UI) cannot access these details.

Code:

```
// File: com/company/hr/Appraisal.java
package com.company.hr;

class Appraisal {  // no modifier = package-private
   String employeeId;
   int rating;
   String comments;

   Appraisal(String id, int rating, String comments) {
      this.employeeId = id;
      this.rating = rating;
      this.comments = comments;
   }

   void showAppraisal() {
      System.out.println("Appraisal for " + employeeId +
         " | Rating: " + rating +
         " | Comments: " + comments);
   }
}

// File: com/company/hr/HrPortal.java
package com.company.hr;

public class HrPortal {
   public static void main(String[] args) {
      Appraisal a = new Appraisal("E101", 5, "Excellent work");
      a.showAppraisal(); // allowed (same package)
   }
```

```
}

// File: com/company/payroll/PayrollSystem.java
package com.company.payroll;

import com.company.hr.*;

public class PayrollSystem {
   public static void main(String[] args) {
      //  Cannot access Appraisal - it's package-private
      // Appraisal a = new Appraisal("E101", 5, "Excellent work");
   }
}
```

Reasoning to interviewer:

- By omitting public or protected, we restrict Appraisal to package-private.
- This way, only HR-related classes in com.company.hr can see appraisal details.
- It enforces information hiding across subsystems.

Future scope:

- Add protected getters if managers (in a subclass within HR) need access.
- Use secure services/APIs to share sanitized appraisal summaries with payroll without exposing full comments.

Static and Instance Variables/Methods

Scenario 32: School system, Instance for student details, static for institution name.

A school system stores each student's name/roll individually, but all share the same institution name. How would you use static vs instance?

Answer:

- Each student has instance fields for name and rollNo (different per object).
- All students belong to the same institution (e.g., "Delhi Public School"), which should be a static field because it's shared by the class, not each object.
- A toString() will combine both.

Code:

```
class Student {
   private final String name;    // instance (unique per student)
   private final int rollNo;     // instance
   public static String institutionName = "Delhi Public School"; // shared by all

   public Student(String name, int rollNo) {
      if (name == null || name.isBlank() || rollNo <= 0)
         throw new IllegalArgumentException("Invalid student data");
      this.name = name.trim();
      this.rollNo = rollNo;
   }

   @Override
   public String toString() {
      return name + " | Roll: " + rollNo + " | School: " + institutionName;
   }

   public static void main(String[] args) {
      Student s1 = new Student("Riya", 1);
      Student s2 = new Student("Karan", 2);
      System.out.println(s1);
      System.out.println(s2);
   }
}
```

Reasoning to interviewer:

- Instance variables (name, rollNo) differ per object.
- Static variable (institutionName) is class-level: changing it once updates it for all students.
- Saves memory and avoids duplication of identical data in each object.

Future scope:

- Make institutionName final if it should never change.
- Store branch info separately if multiple campuses exist.

Scenario 33: School database, Static counter tracks total students

A school database increments a global counter whenever a new student is added. How can a static variable track total students?

Answer:

We want to track how many students have been created in the system overall.

- Use a private static int counter in Student.
- Increment it in the constructor.
- Provide a static method getTotalStudents() to retrieve the count.

Code:

```
class StudentRegistry {
   private final String name;
   private final int rollNo;
   private static int totalStudents = 0; // shared counter

   public StudentRegistry(String name, int rollNo) {
      if (name == null || name.isBlank() || rollNo <= 0)
         throw new IllegalArgumentException("Invalid student data");
      this.name = name.trim();
      this.rollNo = rollNo;
      totalStudents++;  // increment every time a new student is created
   }

   public static int getTotalStudents() {
      return totalStudents;
   }

   @Override
   public String toString() {
      return "Student: " + name + " | Roll: " + rollNo;
   }

   public static void main(String[] args) {
      new StudentRegistry("Anita", 1);
      new StudentRegistry("Rajat", 2);
      System.out.println("Total students: " + StudentRegistry.getTotalStudents()); // 2
   }
}
```

Reasoning to interviewer:

- totalStudents is static so all objects share the same variable.
- Each constructor call increments it once → automatic global counter.
- Static method getTotalStudents() allows access without creating an object.

Future scope:

- Track counters per batch/year by moving counters into a Map<Integer, Integer>.
- Use synchronization if multiple threads are creating students concurrently.

Scenario 34: Factory devices, Shared operational state with static fields

A factory has multiple devices. All devices share the same operational state. How can static fields handle this?

Answer:

In a factory, all devices may share the same operational state (running or maintenance).

- Represent each Device with instance fields like id, type.
- Use a static boolean isRunning to represent global state.
- Changing it in one device affects all others.

Code:

```
class Device {
  private final String id;
  private final String type;
  private static boolean isRunning = true; // shared by all devices

  public Device(String id, String type) {
    if (id == null || id.isBlank() || type == null || type.isBlank())
      throw new IllegalArgumentException("Invalid device data");
    this.id = id.trim();
    this.type = type.trim();
  }

  public static void setOperationalState(boolean state) {
    isRunning = state;
  }

  public void showStatus() {
    System.out.println("Device " + id + " (" + type + ") | Running: " + isRunning);
  }
   public static void main(String[] args) {
    Device d1 = new Device("D01", "Conveyor");
    Device d2 = new Device("D02", "Packaging");
    d1.showStatus();
    d2.showStatus();

    // Change factory state to maintenance
    Device.setOperationalState(false);

    d1.showStatus();
```

```
        d2.showStatus();
    }
}
```

Reasoning to interviewer:

- isRunning is static, so changing it once affects all devices uniformly.
- This models shared factory-level state rather than duplicating the flag in each object.
- Encapsulation: provide a static setter (setOperationalState) to control changes.

Future scope:

- Replace boolean with an enum FactoryState { RUNNING, MAINTENANCE, SHUTDOWN }.
- Broadcast events when state changes so devices can perform cleanup.

Scenario 35: Static method, courier tracking ID generator

A courier system generates unique tracking IDs using a static method. How can this be designed?

Answer:

I'll centralize ID creation in a utility with a static method so callers don't need to instantiate anything. A monotonic counter (e.g., AtomicLong) plus a date/prefix ensures readability and uniqueness.

- Use AtomicLong for thread-safe incrementing across threads.
- Expose TrackingIds.next() as the single source of truth.

Code:

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.atomic.AtomicLong;

final class TrackingIds {
   private static final AtomicLong COUNTER = new AtomicLong(1);
   private static final DateTimeFormatter YYYYMMDD =
DateTimeFormatter.ofPattern("yyyyMMdd");
   private TrackingIds() {} // prevent instantiation

   public static String next() {
      long n = COUNTER.getAndIncrement();
      String today = LocalDate.now().format(YYYYMMDD);
      return "CR-" + today + "-" + String.format("%06d", n); // e.g., CR-20250828-000123
   }
}

class Shipment {
   private final String trackingId;
   public Shipment() { this.trackingId = TrackingIds.next(); }
   public String getTrackingId() { return trackingId; }

   public static void main(String[] args) {
      System.out.println(new Shipment().getTrackingId());
      System.out.println(new Shipment().getTrackingId());
   }
}
```

Reasoning to interviewer:

- A static factory (TrackingIds.next()) avoids scattering ID logic across the codebase and is naturally shared.
- AtomicLong gives lock-free thread safety; no need for synchronized.
- Private constructor on TrackingIds enforces utility semantics.

Future scope:

- Swap the counter with a distributed ID (Snowflake, database sequence) without touching callers.
- Add region/warehouse shards to the prefix for analytics and routing.

Scenario 36: Trains, instance train number, static railway zone

In a railway system, every train object has its unique train number, but all share the same railway zone. How will you use static and instance fields?

Answer:

Each train has its own number (instance field), but all trains in the object model belong to the same railway zone (static field).

- trainNumber → instance, unique per object.
- railwayZone → static, shared by all; make it final if it never changes.

Code:

```
class Train {
   private final int trainNumber;          // instance-specific
   public static String railwayZone = "NORTH WESTERN RAILWAY"; // shared across all
trains

   public Train(int trainNumber) {
      if (trainNumber <= 0) throw new IllegalArgumentException("Invalid train
number");
      this.trainNumber = trainNumber;
   }

   public int getTrainNumber() { return trainNumber; }

   @Override
   public String toString() {
      return "Train " + trainNumber + " | Zone: " + railwayZone;
   }

   public static void main(String[] args) {
      Train t1 = new Train(12957);
      Train t2 = new Train(22965);
      System.out.println(t1);
      System.out.println(t2);

      // If zone changes for all (rare), update once:
      Train.railwayZone = "WESTERN RAILWAY";
      System.out.println(t1);
      System.out.println(t2);
   }
}
```

Reasoning to interviewer:

- Instance data varies per object; static data represents a class-level constant shared by all.
- Changing railwayZone once affects all trains, that's exactly the point of static.
- If the zone is immutable, mark it public static final and set via constant.

Future scope:

- Support multiple zones by introducing subclasses or a zone field per train and move the shared bits (like fare policies) to a separate service.
- Keep a static map of zone codes → zone names if you need lookups.

Scenario 37: Gaming; static leaderboard for top player across sessions

In a gaming app, a static leaderboard tracks the top player across all game sessions. How is static useful here?

Answer:

A leaderboard is global to all game sessions. A static field can store the current top player and score.

- When any session submits a result, compare-and-set the global record.
- Use AtomicReference/volatile for thread visibility if multiple threads update it.

Code:

```
import java.util.concurrent.atomic.AtomicReference;

class LeaderboardEntry {
   final String player;
   final int score;
   LeaderboardEntry(String player, int score) { this.player = player; this.score = score; }
   @Override public String toString() { return player + " - " + score; }
}

class Leaderboard {
   private static final AtomicReference<LeaderboardEntry> TOP =
         new AtomicReference<>(new LeaderboardEntry("-", 0));

   public static void submit(String player, int score) {
      if (player == null || player.isBlank() || score < 0) throw new
IllegalArgumentException();
      while (true) {
         LeaderboardEntry current = TOP.get();
         if (score <= current.score) return; // not a new high score
         LeaderboardEntry candidate = new LeaderboardEntry(player.trim(), score);
         if (TOP.compareAndSet(current, candidate)) return; // success
      }
   }

   public static LeaderboardEntry currentTop() {
      return TOP.get();
   }
}

class GameSession {
```

```
    public static void main(String[] args) {
        Leaderboard.submit("Riya", 1500);
        Leaderboard.submit("Karan", 2100);
        Leaderboard.submit("Riya", 2000); // ignored (below 2100)
        System.out.println("Top: " + Leaderboard.currentTop());
    }
}
```

Reasoning to interviewer:

- Static state naturally models a global leaderboard shared by all sessions.
- AtomicReference gives a lock-free and thread-safe way to update top score under contention.
- Clear boundary: gameplay logic calculates scores; the static leaderboard owns the global record.

Future scope:

- Keep a static TopN structure (min-heap) for top 10 scores.
- Periodically snapshot to persistence; rotate daily/weekly leaderboards.
- Add compareAndExchange or use StampedLock if you need read-heavy optimization.

Encapsulation

Scenario 38: Bank; hide balances, expose secure view method (encapsulation)

A bank hides customer balances but provides a secure method to view it. How can encapsulation help?

Answer:

Encapsulation means keeping sensitive state private and exposing only controlled operations. I'll store balance as a private field and expose a secure read method that requires an authorization flag (or role/context object). No public setter for balance; mutations only through domain methods like deposit/withdraw, which enforce rules.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

class BankAccount {
   private final String accountNo;
   private BigDecimal balance; // sensitive

   public BankAccount(String accountNo, BigDecimal opening) {
      if (accountNo == null || accountNo.isBlank() || opening == null || opening.signum()
< 0)
         throw new IllegalArgumentException("Invalid input");
      this.accountNo = accountNo.trim();
      this.balance = opening.setScale(2, RoundingMode.HALF_UP);
   }

   // domain actions control changes; no direct setBalance
   public void deposit(BigDecimal amt) {
      if (amt == null || amt.signum() <= 0) throw new IllegalArgumentException("Invalid
deposit");
      balance = balance.add(amt).setScale(2, RoundingMode.HALF_UP);
   }
   public void withdraw(BigDecimal amt) {
      if (amt == null || amt.signum() <= 0) throw new IllegalArgumentException("Invalid
withdrawal");
      if (balance.compareTo(amt) < 0) throw new IllegalStateException("Insufficient
funds");
      balance = balance.subtract(amt).setScale(2, RoundingMode.HALF_UP);
   }
```

```
    // secure view; caller must be authorized
    public BigDecimal viewBalance(boolean isAuthorized) {
        if (!isAuthorized) throw new SecurityException("Access denied");
        return balance;
    }

    public String getAccountNo() { return accountNo; } // non-sensitive
}
```

Reasoning to interviewer:

- Private balance prevents accidental exposure or arbitrary writes.
- All mutations go through domain methods that enforce invariants.
- The read-path is also controlled; in real systems, I'd accept a UserContext/Role instead of a boolean and audit every access.

Future scope:

- Replace the boolean with RBAC/ABAC checks and an AuditLog.
- Add limits, fraud checks, and multi-factor approval for high-value withdrawals.
- Keep balances in a ledger and compute available balance from postings rather than a single mutable number.

Scenario 39: Medical system; confidential data with getters only (immutability + encapsulation)

A medical system stores confidential health data inside a Client class. Only getters (no setters) should be provided. How do you enforce this?

Answer:

For PHI, I'll make fields private and expose only getters. No setters, and where data is composite (arrays/collections), I'll return defensive copies or unmodifiable views to prevent external mutation.

Code:

```
import java.util.Collections;
import java.util.List;

final class Client {
   private final String id;
   private final String name;
   private final String diagnosis;        // confidential
   private final List<String> allergies;    // defensive handling

   public Client(String id, String name, String diagnosis, List<String> allergies) {
      if (id == null || id.isBlank() || name == null || name.isBlank())
         throw new IllegalArgumentException("id/name required");
      this.id = id.trim();
      this.name = name.trim();
      this.diagnosis = diagnosis == null ? "" : diagnosis.trim();
      this.allergies = allergies == null ? List.of() : List.copyOf(allergies); // immutable
snapshot
   }

   // getters only; no setters
   public String getId()      { return id; }
   public String getName()    { return name; }
   public String getDiagnosis() { return diagnosis; }
   public List<String> getAllergies() { return Collections.unmodifiableList(allergies); }
}
```

Reasoning to interviewer:

- Private fields + no setters → object state cannot be altered after construction.
- List.copyOf + unmodifiableList ensures callers can't mutate internal lists.
- In practice, access to getDiagnosis() would still be gated by service-level authorization; here I'm modeling the class boundary.

Future scope:

- Wrap diagnosis in a value type with confidentiality levels; expose masked summaries to non-clinical roles.
- Encrypt sensitive fields at rest; decrypt only within a narrow, audited service.
- Version medical records (immutable events) instead of updating in place.

Scenario 40: Shopping app; hide payment details, expose masked card via secure method

A shopping app hides payment details but allows retrieval of masked card numbers via a secure method. How can you apply encapsulation here?

Answer:

Payment details must never be publicly readable. I'll store the raw PAN privately (or, better, not store it at all store a token). Expose a getMaskedCard() that returns only the last 4 digits. Only payment services with proper authorization can request the masked value; no API returns full PAN.

Code:

```
final class PaymentProfile {
   private final String token;     // safe token from payment gateway (preferred)
   private final String cardLast4;  // non-sensitive
   private final String scheme;    // VISA/MC/etc.

   // If legacy requires storing PAN, keep it private & encrypted; shown here as absent
by design.
   public PaymentProfile(String token, String cardLast4, String scheme) {
      if (token == null || token.isBlank() || cardLast4 == null || cardLast4.length() != 4)
         throw new IllegalArgumentException("Invalid payment profile");
      this.token = token.trim();
      this.cardLast4 = cardLast4;
      this.scheme = (scheme == null ? "CARD" : scheme.trim());
   }

   public String getMaskedCard(boolean isPaymentService) {
      if (!isPaymentService) throw new SecurityException("Unauthorized");
      // Standard mask: **** **** **** 1234 (store only last4 so nothing sensitive
leaves)
      return "**** **** **** " + cardLast4;
   }

   public String getScheme() { return scheme; }
   public String getToken()  { return token; }
}
```

Reasoning to interviewer:

- Encapsulation hides sensitive payment data; only a masked representation is ever exposed.
- Using gateway tokens avoids storing raw PAN. industry best practice (PCI-DSS).
- Even the masked method is authorization-gated; logs should capture who accessed it and why.

Future scope:

- Introduce a SecurityContext/Role object, not a boolean; add audit trails.
- Support multiple instruments (UPI, cards) behind an interface; return masks per instrument type.
- Rotate tokens and handle vault interactions via a dedicated PCI-compliant service.

Scenario 41: Insurance; premium private, update only after docs verified (encapsulation)

In an insurance system, policy premium is private and can only be updated if the customer's documents are verified. How can encapsulation enforce this?

Answer:

Keep premium private and block any direct setter. Expose a domain method that updates the premium only if the policy is currently "KYC/document-verified". Verification status itself is controlled-not settable by random callers.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

final class Policy {
   private final String policyNo;
   private BigDecimal premium;        // sensitive, never exposed for direct write
   private boolean documentsVerified;   // internal guard flag

   public Policy(String policyNo, BigDecimal premium) {
      if (policyNo == null || policyNo.isBlank() || premium == null || premium.signum() <
0)
         throw new IllegalArgumentException("Invalid policy");
      this.policyNo = policyNo.trim();
      this.premium  = premium.setScale(2, RoundingMode.HALF_UP);
      this.documentsVerified = false;
   }

   // Controlled verification entry point (e.g., invoked by underwriting workflow)
   public void markDocumentsVerified(boolean verified, String actorRole) {
      if (!"UNDERWRITER".equals(actorRole) && !"COMPLIANCE".equals(actorRole))
         throw new SecurityException("Not authorized to verify documents");
      this.documentsVerified = verified;
   }

   // Encapsulated update path with guard
   public void updatePremium(BigDecimal newPremium) {
      if (!documentsVerified) throw new IllegalStateException("Documents not
verified");
      if (newPremium == null || newPremium.signum() < 0)
         throw new IllegalArgumentException("Invalid premium");
      this.premium = newPremium.setScale(2, RoundingMode.HALF_UP);
   }
```

```
   public BigDecimal viewPremium(String actorRole) {
      if (!"CUSTOMER".equals(actorRole) && !"AGENT".equals(actorRole) &&
!"UNDERWRITER".equals(actorRole))
         throw new SecurityException("Not authorized to view premium");
      return premium;
   }
}
```

Reasoning to interviewer:

- Encapsulation keeps premium private; only a guarded method can change it.
- The “documents verified” flag is also encapsulated; only privileged roles can flip it.
- Callers can’t bypass rules because there’s no public setter-only domain methods with checks.

Future scope:

- Replace role string with a UserContext and audit every premium change.
- Derive premium via a RatingEngine service (age, risk, add-ons) instead of direct assignment.
- Make verification time-bound (auto-revert if KYC expires).

Scenario 42: Banking; ATM PIN hidden; updates only after OTP verification

In a banking app, ATM PIN must never be exposed but should allow updates only after OTP verification. How will you design this?

Answer:

Never store or expose the raw PIN. Keep a salted hash privately. To change the PIN, require an OTP check through a verified service. Encapsulation ensures all updates pass through a guarded method; no getters for PIN exist.

Code:

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Arrays;

final class OtpService {
  public boolean verify(String userId, String otp) {
    // stub: call real OTP provider / cache
    return otp != null && otp.equals("123456");
  }
}

final class AtmProfile {
  private final String userId;
  private final byte[] salt;       // per-user salt
  private byte[] pinHash;          // hashed PIN (never store raw)

  public AtmProfile(String userId, String initialPin) {
    if (userId == null || userId.isBlank() || initialPin == null || initialPin.length() < 4)
      throw new IllegalArgumentException("Invalid data");
    this.userId = userId.trim();
    this.salt = new byte[16];
    new SecureRandom().nextBytes(salt);
    this.pinHash = hash(initialPin, salt);
  }

  // No getter for PIN. Only a guarded update.
  public void changePin(String newPin, String otp, OtpService otpService) {
    if (newPin == null || newPin.length() < 4) throw new
IllegalArgumentException("Weak PIN");
    if (otpService == null || !otpService.verify(userId, otp))
      throw new SecurityException("OTP verification failed");
    this.pinHash = hash(newPin, salt);
```

```
  }

  // Optional: verify entered PIN during ATM auth
  public boolean verifyPin(String enteredPin) {
    return Arrays.equals(pinHash, hash(enteredPin, salt));
  }

  private static byte[] hash(String pin, byte[] salt) {
    try {
      MessageDigest md = MessageDigest.getInstance("SHA-256");
      md.update(salt);
      return md.digest(pin.getBytes(StandardCharsets.UTF_8));
    } catch (Exception e) {
      throw new IllegalStateException("Hashing failed", e);
    }
  }
}
```

Reasoning to interviewer:

- Encapsulation: no PIN getters, no direct mutation; only changePin with OTP guard.
- Security: store only salted hashes; even a DB leak doesn't reveal the actual PIN.
- Separation of concerns: OTP verification is delegated to a service; the profile only enforces the gate.

Future scope:

- Add retry limits, cooldowns, and device binding for PIN changes.
- Use stronger KDFs (PBKDF2/BCrypt/Argon2) instead of raw SHA-256 for PIN hashing.
- Require 2FA factors (OTP + old PIN) and keep an audit trail of changes.

Abstraction

Scenario 43: Courses; abstract base enforces pricing contract (Virtual vs Classroom)

An online learning site has both virtual and classroom courses, but pricing differs. How can an abstract base class enforce a contract?

Answer:

I'll define an abstract Course with common fields (id, title, basePrice) and an abstract finalPrice() that each subtype must implement.

- VirtualCourse may add platform/service fees or apply lower overhead.
- ClassroomCourse adds venue cost, instructor day-rate, seat capacity prorations.
  Abstract class guarantees a consistent API while letting pricing vary per type.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

abstract class Course {
  protected final String id;
  protected final String title;
  protected final BigDecimal basePrice;

  protected Course(String id, String title, BigDecimal basePrice) {
    if (id == null || id.isBlank() || title == null || title.isBlank() || basePrice == null ||
basePrice.signum() < 0)
      throw new IllegalArgumentException("Invalid course data");
    this.id = id.trim();
    this.title = title.trim();
    this.basePrice = basePrice.setScale(2, RoundingMode.HALF_UP);
  }
  public abstract BigDecimal finalPrice();      // pricing contract
  public String summary() { return "%s (%s) => ₹%s".formatted(title,
getClass().getSimpleName(), finalPrice()); }
}

class VirtualCourse extends Course {
  public VirtualCourse(String id, String title, BigDecimal basePrice) { super(id, title,
basePrice); }
  @Override public BigDecimal finalPrice() {
    // Example: lower overhead + platform fee 5%
```

```
    BigDecimal platformFee = basePrice.multiply(new BigDecimal("0.05"));
    return basePrice.add(platformFee).setScale(2, RoundingMode.HALF_UP);
  }
}

class ClassroomCourse extends Course {
  private final BigDecimal venueCostPerSeat;
  private final int seats;
  public ClassroomCourse(String id, String title, BigDecimal basePrice, BigDecimal
venueCostPerSeat, int seats) {
    super(id, title, basePrice);
    if (venueCostPerSeat == null || venueCostPerSeat.signum() < 0 || seats <= 0)
      throw new IllegalArgumentException("Invalid venue/seats");
    this.venueCostPerSeat = venueCostPerSeat.setScale(2,
RoundingMode.HALF_UP);
    this.seats = seats;
  }
  @Override public BigDecimal finalPrice() {
    // Example: allocate venue cost per seat + 12% tax
    BigDecimal seatOverhead = venueCostPerSeat;
    BigDecimal beforeTax = basePrice.add(seatOverhead);
    return beforeTax.add(beforeTax.multiply(new BigDecimal("0.12"))).setScale(2,
RoundingMode.HALF_UP);
  }
}

class CourseDemo {
  public static void main(String[] args) {
    Course v = new VirtualCourse("C101","Java Streams", new BigDecimal("1499"));
    Course c = new ClassroomCourse("C201","Spring Boot Bootcamp", new
BigDecimal("4999"), new BigDecimal("350"), 30);
    System.out.println(v.summary());
    System.out.println(c.summary());
  }
}
```

Reasoning to interviewer:

- Abstract class captures shared identity/state and mandates finalPrice().
- Subclasses can't forget pricing-compiler forces implementation.
- Consumers use Course polymorphically and don't branch on type.

Future scope:

- Add discounts/coupons via a PriceRule strategy injected into Course.
- Introduce HybridCourse later without changing any callers.
- Add taxPolicy per region for compliance.

Scenario 44: Logistics; abstract transport fare (Coach, Locomotive, Aircraft)

A logistics company has Coach, Locomotive, Aircraft each computing fares differently. How would abstraction help?

Answer:

Create an abstract Transit with common fields (distanceKm, weightKg) and abstract fare().

- **Coach (bus) →** base per-km + weight surcharge.
- **Locomotive (train) →** slabbed distance + freight factor.
- **Aircraft →** per-km high rate + fuel surcharge + security fee.
  Abstraction keeps the contract uniform while allowing custom formulas.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

abstract class Transit {
  protected final double distanceKm;
  protected final double weightKg;
  protected Transit(double distanceKm, double weightKg) {
    if (distanceKm <= 0 || weightKg < 0) throw new IllegalArgumentException("Invalid
trip params");
    this.distanceKm = distanceKm; this.weightKg = weightKg;
  }
  public abstract BigDecimal fare();
  protected BigDecimal bd(double v) { return BigDecimal.valueOf(v).setScale(2,
RoundingMode.HALF_UP); }
}

class Coach extends Transit {
  public Coach(double distanceKm, double weightKg) { super(distanceKm,
weightKg); }
  @Override public BigDecimal fare() {
    double base = 2.5 * distanceKm;          // ₹2.5 per km
    double weight = 0.1 * weightKg;          // small luggage fee
    return bd(base + weight);
  }
}

class Locomotive extends Transit {
```

```
  public Locomotive(double distanceKm, double weightKg) { super(distanceKm,
weightKg); }
  @Override public BigDecimal fare() {
    double perKm = distanceKm <= 500 ? 1.8 : 1.5;   // cheaper beyond 500 km
    double freight = 0.05 * weightKg;           // wagon freight factor
    return bd(distanceKm * perKm + freight);
  }
}

class Aircraft extends Transit {
  public Aircraft(double distanceKm, double weightKg) { super(distanceKm,
weightKg); }
  @Override public BigDecimal fare() {
    double base = 7.5 * distanceKm;             // high per-km
    double fuel = 0.12 * base;                  // fuel surcharge
    double sec  = 350.0;                        // flat security fee
    double baggage = Math.max(0, weightKg - 15) * 30; // paid baggage beyond free
15kg
    return bd(base + fuel + sec + baggage);
  }
}

class LogisticsDemo {
  public static void main(String[] args) {
    Transit a = new Coach(120, 20);
    Transit b = new Locomotive(750, 200);
    Transit c = new Aircraft(900, 23);
    System.out.println("Coach fare: ₹" + a.fare());
    System.out.println("Train fare: ₹" + b.fare());
    System.out.println("Air fare:   ₹" + c.fare());
  }
}
```

Reasoning to interviewer:

- Abstract fare() enforces a single computation contract; subclasses plug in formulas.
- Shared validation/state lives in Transit.
- Consumers treat all transports uniformly (List<Transit>), relying on runtime polymorphism.

Future scope:

- Move rates to config/DB and inject a RateCard.
- Add surge pricing or promotions via a FareDecorator.
- Expose estimate() vs finalFare() to include taxes/airport fees later.

Scenario 45: Payments; common interface for CreditCard, UPI, NetBanking

A payment gateway supports CreditCardPayment, UPIPayment, NetBankingPayment. All must follow the same interface for processing. How do you design this?

Answer:

Use an interface Payment declaring validate(), authorize(amount), and capture(amount).

- CreditCardPayment talks to a card gateway (token, CVV rules).
- UPIPayment validates VPA and polls collect requests.
- NetBankingPayment redirects to bank and confirms callback.
  Polymorphism lets a checkout pipeline accept any Payment without if/else blocks.

Code:

```
import java.math.BigDecimal;

interface Payment {
   void validate();                  // instrument-level checks
   boolean authorize(BigDecimal amount);    // hold funds / create intent
   boolean capture(BigDecimal amount);     // finalize charge
   default String name() { return getClass().getSimpleName(); }
}

final class CreditCardPayment implements Payment {
   private final String token; // from PCI-compliant vault
   public CreditCardPayment(String token) {
      if (token == null || token.isBlank()) throw new IllegalArgumentException("Invalid
token");
      this.token = token.trim();
   }
   @Override public void validate() { /* Luhn/token checks via gateway */ }
   @Override public boolean authorize(BigDecimal amount) { /* call gateway auth */
return true; }
   @Override public boolean capture(BigDecimal amount) { /* capture auth */ return
true; }
}

final class UPIPayment implements Payment {
   private final String vpa; // e.g., user@okicici
   public UPIPayment(String vpa) {
```

```
    if (vpa == null || !vpa.contains("@")) throw new IllegalArgumentException("Invalid
VPA");
    this.vpa = vpa.trim();
  }
  @Override public void validate() { /* ping UPI PSP for VPA existence */ }
  @Override public boolean authorize(BigDecimal amount) { /* send collect request
*/ return true; }
  @Override public boolean capture(BigDecimal amount) { /* confirm success */
return true; }
}

final class NetBankingPayment implements Payment {
  private final String bankCode; // e.g., HDFC, SBI
  public NetBankingPayment(String bankCode) {
    if (bankCode == null || bankCode.isBlank()) throw new
IllegalArgumentException("Invalid bank");
    this.bankCode = bankCode.trim();
  }
  @Override public void validate() { /* check supported banks */ }
  @Override public boolean authorize(BigDecimal amount) { /* redirect/init session
*/ return true; }
  @Override public boolean capture(BigDecimal amount) { /* confirm callback status
*/ return true; }
}

class Checkout {
  public static boolean process(Payment p, BigDecimal amount) {
    p.validate();
    if (!p.authorize(amount)) return false;
    return p.capture(amount);
  }
  public static void main(String[] args) {
    Payment cc  = new CreditCardPayment("tok_abc123");
    Payment upi = new UPIPayment("rajat@okhdfcbank");
    Payment nb  = new NetBankingPayment("HDFC");
    System.out.println(cc.name()  + " => " + process(cc,  new
BigDecimal("1499.00")));
    System.out.println(upi.name() + " => " + process(upi, new BigDecimal("299.00")));
    System.out.println(nb.name()  + " => " + process(nb,  new
BigDecimal("2499.00")));
  }
}
```

Reasoning to interviewer:

- The Payment interface is the contract; each method is a lifecycle step.
- Concrete classes encapsulate provider-specific details; checkout code is decoupled and testable.
- Adding WalletPayment later means just another class implementing Payment. no change to Checkout.

Future scope:

- Add refund() to the interface; support partial captures.
- Integrate idempotency keys and retries for network resilience.
- Use dependency injection to supply gateways/middleware (3-DS, risk checks) per payment type.

Scenario 46: Ride-hailing; abstract base enforces fare contract (CarRide, BikeRide, AutoRide)

A ride-hailing service (CarRide, BikeRide, AutoRide) must calculate fares differently. How would abstraction help?

Answer:

I'll define an abstract Ride capturing shared state (distanceKm, waitMinutes, surgeMultiplier) and an abstract fare() method. Each concrete ride type implements its own pricing formula. Consumers work with Ride references and call fare() without knowing the type.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

abstract class Ride {
   protected final double distanceKm;
   protected final int waitMinutes;
   protected final BigDecimal surge; // e.g., 1.0 = normal, 1.5 = 50% surge

   protected Ride(double distanceKm, int waitMinutes, BigDecimal surge) {
      if (distanceKm <= 0 || waitMinutes < 0 || surge == null || surge.signum() <= 0)
         throw new IllegalArgumentException("Invalid ride params");
      this.distanceKm = distanceKm;
      this.waitMinutes = waitMinutes;
      this.surge = surge;
   }

   public abstract BigDecimal fare(); // contract every ride must implement

   protected BigDecimal bd(double v) {
      return BigDecimal.valueOf(v).setScale(2, RoundingMode.HALF_UP);
   }
}

class CarRide extends Ride {
   public CarRide(double km, int waitMin, BigDecimal surge) { super(km, waitMin,
surge); }
   @Override public BigDecimal fare() {
      // base ₹50 + ₹12/km + ₹2/min wait, then apply surge
      BigDecimal base = bd(50 + 12 * distanceKm + 2 * waitMinutes);
      return base.multiply(surge).setScale(2, RoundingMode.HALF_UP);
   }
```

```
}

class BikeRide extends Ride {
  public BikeRide(double km, int waitMin, BigDecimal surge) { super(km, waitMin,
surge); }
  @Override public BigDecimal fare() {
    // cheaper per km, minimal wait charge
    BigDecimal base = bd(20 + 7.5 * distanceKm + 0.5 * waitMinutes);
    return base.multiply(surge).setScale(2, RoundingMode.HALF_UP);
  }
}

class AutoRide extends Ride {
  public AutoRide(double km, int waitMin, BigDecimal surge) { super(km, waitMin,
surge); }
  @Override public BigDecimal fare() {
    // meter minimum + per km + fixed auto levy
    BigDecimal base = bd(30 + 10 * distanceKm + 5); // ₹5 levy
    // first 3 minutes free
    BigDecimal waitFee = waitMinutes > 3 ? bd(1.0 * (waitMinutes - 3)) : bd(0);
    return base.add(waitFee).multiply(surge).setScale(2, RoundingMode.HALF_UP);
  }
}

class RideDemo {
  public static void main(String[] args) {
    Ride car  = new CarRide(8.2, 5, new BigDecimal("1.2"));
    Ride bike = new BikeRide(4.0, 2, new BigDecimal("1.0"));
    Ride auto = new AutoRide(6.5, 7, new BigDecimal("1.0"));
    System.out.println("Car:  ₹" + car.fare());
    System.out.println("Bike: ₹" + bike.fare());
    System.out.println("Auto: ₹" + auto.fare());
  }
}
```

Reasoning to interviewer:

- Abstraction centralizes common fields/validation and forces a fare() implementation. no ride can “forget” pricing.
- Polymorphism: List<Ride> can be priced uniformly; runtime dispatch picks the correct formula.
- Using BigDecimal avoids currency precision bugs.

Future scope:

- Move rates into a RateCard service and inject it, enabling locality/time-based pricing.
- Add promotions via a decorator/price-rule pipeline before returning fare().
- Distinguish estimate() vs finalFare() to add tolls/taxes at the end.

Scenario 47: File processing; abstract parse() for CSVFile, XMLFile, JSONFile

A file-processing system (CSVFile, XMLFile, JSONFile) all must implement a parse() method differently. How will you use abstraction?

Answer:

I'll define an abstraction that mandates parse() while letting each file type handle its format. For a minimal example, I'll return a Map<String, Object> or List<Map<...>>. The base class carries the file path and common validation; subclasses implement parse().

Code:

```
import java.io.*;
import java.util.*;

abstract class DataFile {
   protected final String path;
   protected DataFile(String path) {
      if (path == null || path.isBlank()) throw new IllegalArgumentException("Path
required");
      this.path = path.trim();
   }
   public abstract Object parse() throws IOException; // contract
   protected BufferedReader reader() throws FileNotFoundException {
      return new BufferedReader(new FileReader(path));
   }
}

class CSVFile extends DataFile {
   public CSVFile(String path) { super(path); }
   @Override public List<Map<String,String>> parse() throws IOException {
      try (BufferedReader br = reader()) {
         String header = br.readLine();
         if (header == null) return List.of();
         String[] cols = header.split(",");
         List<Map<String,String>> rows = new ArrayList<>();
         for (String line; (line = br.readLine()) != null; ) {
            String[] vals = line.split(",", -1);
            Map<String,String> row = new LinkedHashMap<>();
            for (int i = 0; i < cols.length; i++) {
               row.put(cols[i].trim(), i < vals.length ? vals[i].trim() : "");
            }
            rows.add(row);
         }
         return rows;
```

```
    }
  }
}

class XMLFile extends DataFile {
  public XMLFile(String path) { super(path); }
  @Override public Map<String,Object> parse() throws IOException {
    // Minimal illustrative parse (pseudo): real code would use a proper XML parser
(JAXP)
    Map<String,Object> result = new LinkedHashMap<>();
    try (BufferedReader br = reader()) {
      StringBuilder sb = new StringBuilder();
      for (String line; (line = br.readLine()) != null; ) sb.append(line);
      String xml = sb.toString();
      // pretend: extract root tag name
      String root = xml.replaceAll(".*<([a-zA-Z0-9_:-]+)[^>]*>.*", "$1");
      result.put("root", root);
      result.put("length", xml.length());
      return result;
    }
  }
}

class JSONFile extends DataFile {
  public JSONFile(String path) { super(path); }
  @Override public Map<String,Object> parse() throws IOException {
    // Minimal illustrative parse: real code should use Jackson/Gson
    Map<String,Object> result = new LinkedHashMap<>();
    try (BufferedReader br = reader()) {
      StringBuilder sb = new StringBuilder();
      for (String line; (line = br.readLine()) != null; ) sb.append(line.trim());
      String json = sb.toString();
      result.put("startsWithBrace", json.startsWith("{"));
      result.put("length", json.length());
      return result;
    }
  }
}

class ParserDemo {
  public static void main(String[] args) throws Exception {
    DataFile csv  = new CSVFile("students.csv");
    DataFile xml  = new XMLFile("config.xml");
    DataFile json = new JSONFile("data.json");
    // Polymorphic calls
    System.out.println(csv.parse());
    System.out.println(xml.parse());
```

```
        System.out.println(json.parse());
    }
}
```

Reasoning to interviewer:

- Abstraction (DataFile) defines the parse() contract; each concrete class handles its format specifics.
- Common concerns (path validation, opening readers) live in the base.
- Callers can keep a List<DataFile> and parse all files polymorphically without instanceof ladders.

Future scope:

- Replace illustrative parsing with libraries (Jackson for JSON, JAXP/StAX for XML, OpenCSV).
- Stream large files (line by line) to avoid memory spikes; return iterators/streams instead of materialized lists.
- Add a Schema/Validator layer for strict validation and better error reporting.

Polymorphism (Compile-Time & Runtime)

Scenario 48: Calculator; overloaded multiply(int, float, double) = compile-time polymorphism

A Calculator class supports multiple multiply() methods (int, float, double). How is this compile-time polymorphism?

Answer:

This is method overloading: same method name, different parameter types. The compiler decides which multiply() to bind based on the argument types at compile time (no runtime dispatch). If needed, Java performs widening (e.g., int → long → float → double) to pick the most specific match.

Code:

```
class Calculator {
   public int multiply(int a, int b) {         // exact for ints
      return a * b;
   }
   public float multiply(float a, float b) {     // exact for floats
      return a * b;
   }
   public double multiply(double a, double b) {   // exact for doubles
      return a * b;
   }

   public static void main(String[] args) {
      Calculator c = new Calculator();

      System.out.println(c.multiply(3, 4));        // picks (int,int)
      System.out.println(c.multiply(3.5f, 2.0f));    // picks (float,float)
      System.out.println(c.multiply(3.5, 2.0));     // picks (double,double)

      // Widening example:
      System.out.println(c.multiply(5, 2.5));       // (int,double) → matches
(double,double)
      System.out.println(c.multiply(5, 2.5f));      // (int,float)  → matches (float,float)
   }
}
```

Reasoning to interviewer:

- **Overloading** = compile-time polymorphism; resolution happens by signature during compilation.
- The **"most specific applicable"** method wins; otherwise Java widens types.
- No runtime dispatch here-invokevirtual still calls the exact method resolved at compile time.

Future scope:

- Add multiply(int... nums) (varargs) for N-ary multiplication.
- Guard against overflow using Math.multiplyExact() for integer variants.

Scenario 49: Streaming; Regular vs Premium override playContent() = runtime polymorphism

A streaming service has Regular and Premium users, each overriding playContent(). How does runtime polymorphism work here?

Answer:

Define a base User with an abstract playContent(title). RegularUser overrides to play with ads and SD quality; PremiumUser overrides for ad-free HD/4K. At runtime, a User reference points to different concrete objects, and the JVM dispatches to the correct playContent() implementation.

Code:

```
abstract class User {
   protected final String name;
   protected User(String name) { this.name = name; }
   public abstract void playContent(String title);
}

class RegularUser extends User {
   public RegularUser(String name) { super(name); }
   @Override public void playContent(String title) {
      System.out.println("[Regular] " + name + " playing " + title + " in SD with ads");
   }
}

class PremiumUser extends User {
   public PremiumUser(String name) { super(name); }
   @Override public void playContent(String title) {
      System.out.println("[Premium] " + name + " playing " + title + " in 4K, no ads, Dolby
Atmos");
   }
}

class StreamingDemo {
   public static void main(String[] args) {
      User u1 = new RegularUser("Riya");
      User u2 = new PremiumUser("Karan");

      // Runtime polymorphism: same call, different behavior
      u1.playContent("Planet Earth");
      u2.playContent("Planet Earth");
   }
}
```

Reasoning to interviewer:

- Overriding implements runtime polymorphism, the method that runs depends on the object's actual type, not the reference type.
- This keeps the player pipeline type-agnostic: it just calls playContent().
- Easy to add FamilyPremiumUser later without touching the call sites.

Future scope:

- Introduce a StreamPolicy (bitrate, DRM) injected per user type.
- Add parental controls by overriding pre-check hooks in subclasses.

Scenario 50: Enterprise modules; FinanceModule & LogisticsModule implement ComputationUnit (interface polymorphism)

An enterprise has FinanceModule and LogisticsModule implementing ComputationUnit. How is polymorphism demonstrated here?

Answer:

Create a common interface ComputationUnit with compute(Order). FinanceModule computes taxes/overheads; LogisticsModule computes freight/shipping. Client code holds a List<ComputationUnit> and invokes compute() uniformly-classic interface-based polymorphism.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

class Order {
   final BigDecimal base;   // merchandise value
   final double distanceKm; // for logistics
   final double weightKg;
   Order(BigDecimal base, double distanceKm, double weightKg) {
      this.base = base; this.distanceKm = distanceKm; this.weightKg = weightKg;
   }
}

interface ComputationUnit {
   BigDecimal compute(Order o);
   default String name() { return getClass().getSimpleName(); }
}

class FinanceModule implements ComputationUnit {
   @Override public BigDecimal compute(Order o) {
      BigDecimal tax = o.base.multiply(new BigDecimal("0.18"));
      BigDecimal overhead = o.base.multiply(new BigDecimal("0.015"));
      return o.base.add(tax).add(overhead).setScale(2, RoundingMode.HALF_UP);
   }
}

class LogisticsModule implements ComputationUnit {
   @Override public BigDecimal compute(Order o) {
      BigDecimal kmCost = new
BigDecimal("8.75").multiply(BigDecimal.valueOf(o.distanceKm));
```

```
    BigDecimal weightCost = new
BigDecimal("1.80").multiply(BigDecimal.valueOf(o.weightKg));
    return kmCost.add(weightCost).setScale(2, RoundingMode.HALF_UP);
  }
}

class Engine {
  public static void main(String[] args) {
    Order order = new Order(new BigDecimal("12000"), 320, 85);
    List<ComputationUnit> units = List.of(new FinanceModule(), new
LogisticsModule());

    units.forEach(u -> System.out.println(u.name() + " => ₹" + u.compute(order)));
  }
}
```

Reasoning to interviewer:

- Interface defines the contract; each module provides its own implementation.
- Client code is decoupled from concrete classes, works with ComputationUnit references.
- Adding InsuranceModule or ComplianceModule is a drop-in change; the loop doesn't change.

Future scope:

- Build a ComputationPipeline that aggregates/weights results from multiple units.
- Externalize rates to config; inject using DI for environment-specific behavior.
- Add caching decorators for expensive computations.

Scenario 51: Gaming app, characters override attack() differently (runtime polymorphism)

In a gaming app, different characters (Warrior, Archer, Mage) implement the same attack() method but behave differently. How does polymorphism apply?

Answer:

Define a common Character base class with an abstract attack().

- Warrior does melee sword strikes.
- Archer shoots arrows with range advantage.
- Mage casts spells consuming mana.
  Polymorphism ensures the same call attack() executes different logic at runtime depending on the actual object.

Code:

```
abstract class Character {
   protected final String name;
   public Character(String name) { this.name = name; }
   public abstract void attack();
}

class Warrior extends Character {
   public Warrior(String name) { super(name); }
   @Override public void attack() {
      System.out.println(name + " swings a sword with brute force!");
   }
}

class Archer extends Character {
   public Archer(String name) { super(name); }
   @Override public void attack() {
      System.out.println(name + " fires a precise arrow from distance!");
   }
}

class Mage extends Character {
   public Mage(String name) { super(name); }
   @Override public void attack() {
      System.out.println(name + " casts a fireball using mana!");
   }
}

class GameDemo {
```

```
public static void main(String[] args) {
    Character[] party = {
        new Warrior("Thor"),
        new Archer("Robin"),
        new Mage("Merlin")
    };
    for (Character c : party) {
        c.attack(); // same call → different behavior
    }
}
```

Reasoning to interviewer:

- The attack() contract is common, but runtime polymorphism dispatches behavior based on object type.
- This allows extensibility: adding Assassin or Healer means just another class with its own attack().
- Game loop code stays clean, it doesn't care which subclass it is.

Future scope:

- Add shared stats (HP, Mana) in base class.
- Apply design patterns like Strategy for dynamic skill selection.

Scenario 52: Hospital system; Doctor, Nurse, Surgeon override performDuty() differently

In a hospital system, Doctor, Nurse, and Surgeon all override a performDuty() method differently. How does polymorphism apply?

Answer:

An abstract Staff class has performDuty(). Subclasses override:

- Doctor prescribes treatments.
- Nurse administers medicine.
- Surgeon performs operations.
  This demonstrates polymorphism in professional roles.

Code:

```
abstract class Staff {
   protected final String name;
   public Staff(String name) { this.name = name; }
   public abstract void performDuty();
}

class Doctor extends Staff {
   public Doctor(String name) { super(name); }
   @Override public void performDuty() {
      System.out.println("Dr. " + name + " diagnoses patients and prescribes
treatment.");
   }
}

class Nurse extends Staff {
   public Nurse(String name) { super(name); }
   @Override public void performDuty() {
      System.out.println("Nurse " + name + " administers medicine and monitors
vitals.");
   }
}

class Surgeon extends Staff {
   public Surgeon(String name) { super(name); }
   @Override public void performDuty() {
      System.out.println("Surgeon " + name + " performs surgical operations in the
OR.");
   }
}
```

```
class HospitalDemo {
   public static void main(String[] args) {
      Staff[] team = {
         new Doctor("Meera"),
         new Nurse("Sita"),
         new Surgeon("Ramesh")
      };
      for (Staff s : team) {
         s.performDuty(); // one method call, different duties
      }
   }
}
```

Reasoning to interviewer:

- Polymorphism lets us treat all staff members uniformly while delegating duties.
- Easy to extend: adding Physiotherapist or Radiologist requires no change to caller code.
- Hospital workflows can loop over a generic Staff[] instead of writing role-specific logic everywhere.

Future scope:

- Enforce scheduling and shift management by introducing abstract hooks like startShift().
- Attach Specialization strategy for dynamic duties.

Scenario 53: E-commerce checkout; runtime polymorphism in payment (Card, Wallet, COD)

In an e-commerce checkout, payment is processed differently depending on whether it's Card, Wallet, or CashOnDelivery. How is runtime polymorphism applied?

Answer:

Define a PaymentMethod interface with a pay(amount) method.

- CardPayment processes via gateway.
- WalletPayment deducts from stored credits.
- CashOnDelivery flags payment pending until delivery.
  At checkout, the system holds a PaymentMethod reference and calls pay() without knowing the actual type, runtime polymorphism.

Code:

```
import java.math.BigDecimal;

interface PaymentMethod {
   boolean pay(BigDecimal amount);
}

class CardPayment implements PaymentMethod {
   @Override public boolean pay(BigDecimal amount) {
      System.out.println("Processing card payment of ₹" + amount);
      return true; // assume gateway success
   }
}

class WalletPayment implements PaymentMethod {
   private BigDecimal balance = new BigDecimal("1000");
   @Override public boolean pay(BigDecimal amount) {
      if (balance.compareTo(amount) < 0) {
         System.out.println("Wallet insufficient balance!");
         return false;
      }
      balance = balance.subtract(amount);
      System.out.println("Wallet payment successful. Remaining: ₹" + balance);
      return true;
   }
}

class CashOnDelivery implements PaymentMethod {
   @Override public boolean pay(BigDecimal amount) {
```

```
        System.out.println("COD selected. Collect ₹" + amount + " on delivery.");
        return true;
    }
}

class CheckoutDemo {
    public static void main(String[] args) {
        PaymentMethod[] methods = {
            new CardPayment(),
            new WalletPayment(),
            new CashOnDelivery()
        };
        for (PaymentMethod m : methods) {
            m.pay(new BigDecimal("750"));
        }
    }
}
```

Reasoning to interviewer:

- PaymentMethod defines the contract; each type overrides pay() with its own rules.
- Runtime polymorphism: same pay() call → gateway debit, wallet deduction, or COD flagging.
- Checkout flow doesn't change when adding new methods (UPI, EMI, Crypto, etc.).

Future scope:

- Add validation (validate() method in interface).
- Integrate fraud/risk checks before final pay().
- Use dependency injection to plug in new payment types dynamically.

Final Keyword & Constants

Scenario 54: Student portal; maximum subjects per semester must never change (use final)

In a student portal, the maximum number of subjects per semester should never change. How do you enforce this using final?

Answer:

I'll declare the maxSubjectsPerSemester as a final variable inside the Student (or SemesterRules) class.

- Since final variables can only be assigned once, the maximum is fixed.
- Any attempt to reassign it will cause a compile-time error.
- This enforces the rule that the maximum subjects value is immutable.

Code:

```
class SemesterRules {
   private final int maxSubjectsPerSemester;  // fixed, immutable

   public SemesterRules(int maxSubjects) {
      if (maxSubjects <= 0) throw new IllegalArgumentException("Invalid max
subjects");
      this.maxSubjectsPerSemester = maxSubjects; // assigned only once
   }

   public int getMaxSubjects() {
      return maxSubjectsPerSemester;
   }

   public static void main(String[] args) {
      SemesterRules rules = new SemesterRules(6);
      System.out.println("Max subjects allowed: " + rules.getMaxSubjects());
      // rules.maxSubjectsPerSemester = 8; // ❌ compile-time error
   }
}
```

Reasoning to interviewer:

- final ensures immutability, the rule can’t change mid-program.
- Prevents bugs like accidentally updating the cap after students are enrolled.

Future scope:

- Move this into a config or database table if institutions differ.
- Combine with static if the rule is universal for all students globally.

Scenario 55: Tax system; GST constant across application (use static final)

A tax rate (GST) must be constant across the application. How would you use static final?

Answer:

GST is a universal constant. I'll declare it as public static final double GST_RATE.

- static ensures it belongs to the class, not instances.
- final ensures immutability.
- By convention, constants are written in UPPER_SNAKE_CASE.

Code:

```
class TaxConfig {
   public static final double GST_RATE = 0.18; // 18% GST constant

   public static double applyGST(double amount) {
      return amount + amount * GST_RATE;
   }

   public static void main(String[] args) {
      double price = 1000;
      System.out.println("Price with GST: " + applyGST(price));
   }
}
```

Reasoning to interviewer:

- static final = compile-time constant accessible globally.
- Only one copy exists in memory, shared across the app.
- Prevents accidental modification (immutable).

Future scope:

- Extract to a config file if GST may vary by country.
- Add more constants like IGST, CGST, SGST with regional applicability.

Scenario 56: Banking; make UserAccount final to prevent subclassing

In banking, UserAccount should not be extended by any class for security. How does final class help?

Answer:

In security-sensitive systems, we may want to forbid inheritance to prevent misuse.

- By marking the class final, no one can extend it.
- This prevents attackers from subclassing and overriding sensitive methods (like withdraw()).

Code:

```
final class UserAccount {
   private final String accountNo;
   private double balance;

   public UserAccount(String accountNo, double openingBalance) {
      if (accountNo == null || accountNo.isBlank() || openingBalance < 0)
         throw new IllegalArgumentException("Invalid data");
      this.accountNo = accountNo.trim();
      this.balance = openingBalance;
   }

   public synchronized void deposit(double amt) {
      if (amt <= 0) throw new IllegalArgumentException("Invalid deposit");
      balance += amt;
   }

   public synchronized boolean withdraw(double amt) {
      if (amt <= 0 || amt > balance) return false;
      balance -= amt;
      return true;
   }

   public double getBalance() { return balance; }
}

// Not allowed:
// class VIPAccount extends UserAccount { } // compile-time error
```

Reasoning to interviewer:

- Declaring UserAccount as final blocks subclassing.
- Ensures core banking logic (deposit, withdraw) can't be overridden with malicious behavior.
- Promotes immutability of business rules in sensitive domains.

Future scope:

- Combine with final methods for extra security in other classes.
- Use composition instead of inheritance if extensions are needed safely.

Scenario 57: Use of this, resolve shadowing and enable fluent setters

A warehouse maintains product categories (TECHNOLOGY, FOOD, APPAREL) as constants. How do you enforce this?

Answer:

When constructor parameters or setters have the same names as fields, this disambiguates the instance fields. I'll also return this from setters to enable fluent chaining.

Code:

```
class StudentProfile {
   private String name;
   private int    roll;

   public StudentProfile(String name, int roll) { // params shadow fields
      this.name = name;   // 'this' refers to the field
      this.roll = roll;
   }

   // fluent updates
   public StudentProfile setName(String name) {
      this.name = name;
      return this;       // chaining
   }
   public StudentProfile setRoll(int roll) {
      this.roll = roll;
      return this;
   }

   @Override public String toString() { return name + " (" + roll + ")"; }

   public static void main(String[] args) {
      StudentProfile s = new StudentProfile("Riya", 12)
                        .setName("Riya Sharma")
                        .setRoll(15);
      System.out.println(s);
   }
}
```

Reasoning to interviewer:

- this.field cleanly distinguishes instance variables from parameters.
- Returning this makes a compact, readable builder-like API without a separate builder class.

Future scope:

- Convert to an immutable object and use a dedicated Builder if many optional fields appear.

Scenario 58: Use of this() constructor chaining to centralize validation

An organization stores team contact info that must remain immutable. How would you use final fields for this?

Answer:

If a class supports defaulting, route lighter constructors to the “primary” one using this(). That centralizes validation and reduces duplication.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;

class Order {
   private final String id;
   private final BigDecimal amount;
   private final String currency;

   // primary constructor (all validation here)
   public Order(String id, BigDecimal amount, String currency) {
      if (id == null || id.isBlank()) throw new IllegalArgumentException("id required");
      if (amount == null || amount.signum() < 0) throw new
IllegalArgumentException("amount >= 0");
      if (currency == null || currency.isBlank()) throw new
IllegalArgumentException("currency required");
      this.id = id.trim();
      this.amount = amount.setScale(2, RoundingMode.HALF_UP);
      this.currency = currency.trim();
   }

   // defaults to INR
   public Order(String id, BigDecimal amount) { this(id, amount, "INR"); }

   // defaults amount to 0.00 INR
   public Order(String id) { this(id, new BigDecimal("0.00"), "INR"); }

   @Override public String toString() { return id + " | " + currency + " " + amount; }
}
```

Reasoning to interviewer:

- this(...) removes duplicated checks and ensures one source of truth for invariants.
- Future changes (rounding, currency rules) happen in one place.

Future scope:

- Add a Builder if optional fields explode (coupon, shipping, notes).

Scenario 59: Use of super; call parent constructor and reuse behaviour

A freight management system has a Lorry object whose load capacity and energy type cannot change once created. How can immutability be achieved?

Answer:

A subclass must initialize base state first. Use super(...) to invoke the parent's constructor and optionally reuse or extend its behavior.

Code:

```
class Employee {
   protected final String id;
   protected final String name;

   public Employee(String id, String name) {
      if (id == null || id.isBlank() || name == null || name.isBlank())
         throw new IllegalArgumentException("Invalid");
      this.id = id.trim(); this.name = name.trim();
   }

   public String role() { return "Employee"; }
}

class Manager extends Employee {
   private final int teamSize;

   public Manager(String id, String name, int teamSize) {
      super(id, name);              // initialize base part first
      if (teamSize < 0) throw new IllegalArgumentException("teamSize");
      this.teamSize = teamSize;
   }

   @Override public String role() { return "Manager"; }

   @Override public String toString() {
      return super.role() + "→" + role() + " " + name + " (team: " + teamSize + ")";
   }
}
```

Reasoning to interviewer:

- **super(...)** guarantees the base class invariants are set before child fields.
- **Overriding role()** shows polymorphism; **super.role()** lets me compose parent + child output.

Future scope:

- Guard creation behind factories if construction becomes complex (policies, privileges).

Scenario 60: Use of super to access hidden members and overridden methods

Answer:

If a subclass overrides a method but needs the parent's original logic, call **super.method().** Also, when a field name is hidden/shadowed, super.field (for non-private, typically protected) accesses the parent version.

Code:

```
class Report {
   protected String header = "Base-Header";
   public String render() { return header + " | content"; }
}

class AuditReport extends Report {
   protected String header = "Audit-Header"; // hides parent field

   @Override public String render() {
      // reuse parent rendering, then add audit extras
      String base = super.render();        // calls Report.render()
      return base + " | auditTrail";
   }

   public String bothHeaders() {
      return "child=" + this.header + ", parent=" + super.header; // access hidden parent
field
   }

   public static void main(String[] args) {
      AuditReport ar = new AuditReport();
      System.out.println(ar.render());
      System.out.println(ar.bothHeaders());
   }
}
```

Reasoning to interviewer:

- super.render() composes behavior instead of duplicating it.
- Accessing super.header demonstrates controlled access to parent state when it's protected (not private).
- Prefer composition if heavy reliance on parent internals grows.

Future scope:

- If the parent must enforce steps, use the Template Method pattern: parent defines the algorithm and calls hook() methods overridden by children.

Scenario 61: Logistics app, enforce MAX_PARCEL_WEIGHT as a constant with final

A logistics app uses MAX_PARCEL_WEIGHT as a constant. How do you enforce this with final?

Answer:

Parcel weight limits must never change across the app. I'll declare MAX_PARCEL_WEIGHT as a public static final constant inside a config or utility class.

- final makes it immutable.
- static ensures it's class-level (shared across all instances).
- By convention, constants are written in UPPER_SNAKE_CASE.

Code:

```
class LogisticsConfig {
   public static final double MAX_PARCEL_WEIGHT = 50.0; // in kilograms

   public static boolean validateWeight(double weight) {
      return weight > 0 && weight <= MAX_PARCEL_WEIGHT;
   }

   public static void main(String[] args) {
      System.out.println("Max parcel weight allowed: " + MAX_PARCEL_WEIGHT +
"kg");
      System.out.println("25kg valid? " + validateWeight(25));  // true
      System.out.println("75kg valid? " + validateWeight(75));  // false
   }
}
```

Reasoning to interviewer:

• Declaring as public static final enforces immutability and ensures there's only one copy in memory.
• If someone tries to reassign MAX_PARCEL_WEIGHT, it fails at compile-time.
• Consumers reference it as LogisticsConfig.MAX_PARCEL_WEIGHT.

Future scope:

• Extract constants to a central Constants class or external config (if rules vary by region).
• Add units (kg, lbs) for multi-country deployments.

Scenario 62: Security framework, make EncryptionEngine final to prevent subclassing

In a security framework, EncryptionEngine class should not be extended by anyone. Why would you make it final?

Answer:

In cryptography, extending or overriding core classes can introduce vulnerabilities (e.g., overriding encrypt() with weaker logic). By making the class final, we prevent inheritance and ensure no one can tamper with critical security logic.

Code:

```
public final class EncryptionEngine {
   private final String algorithm;

   public EncryptionEngine(String algorithm) {
      if (algorithm == null || algorithm.isBlank())
         throw new IllegalArgumentException("Algorithm required");
      this.algorithm = algorithm;
   }

   public byte[] encrypt(byte[] data, byte[] key) {
      // secure encryption logic (stubbed for example)
      System.out.println("Encrypting using " + algorithm);
      return data; // replace with real cipher logic
   }

   public byte[] decrypt(byte[] data, byte[] key) {
      // secure decryption logic
      System.out.println("Decrypting using " + algorithm);
      return data; // replace with real cipher logic
   }
}

// Not allowed:
// class MyEngine extends EncryptionEngine {}  // Compile-time error
```

Reasoning to interviewer:

- Marking the class final means no subclass can override methods.
- This is critical for security classes, attackers can't extend and weaken logic.
- Encourages controlled usage: if new algorithms are needed, add methods or factories inside the same class, not by unsafe subclassing.

Future scope:

- Pair final with private constructors + factory methods for controlled instantiation.
- Consider using sealed (Java 17) if only a whitelisted set of extensions should be allowed.

Inner and Nested Classes

Scenario 63: E-commerce cart, products represented as nested classes

An e-commerce cart has multiple products, each represented as a nested class. How do you design this?

Answer:

I'll keep a Cart as the outer aggregate and model each Product as a static nested class so it doesn't capture the cart instance. If I need product variants (Size, Color), I can add small nested value-types under Product. The cart holds a list of product lines with quantity and can compute totals.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.*;

class Cart {
  // Static nested class: independent data model (no implicit link to Cart)
  public static final class Product {
    private final String sku;
    private final String title;
    private final BigDecimal price;

    public Product(String sku, String title, BigDecimal price) {
      if (sku == null || sku.isBlank() || title == null || title.isBlank()
        || price == null || price.signum() < 0)
        throw new IllegalArgumentException("Invalid product");
      this.sku = sku.trim();
      this.title = title.trim();
      this.price = price.setScale(2, RoundingMode.HALF_UP);
    }
    public String sku()   { return sku; }
    public String title() { return title; }
    public BigDecimal price() { return price; }

    // Example of another tiny nested value-type
    public static final class Size {
      public static final Size S = new Size("S");
      public static final Size M = new Size("M");
      public static final Size L = new Size("L");
      private final String label;
      private Size(String label) { this.label = label; }
```

```
        public String label() { return label; }
    }
  }

  // Cart line item referencing nested Product
  public static final class Line {
    private final Product product;
    private int qty;

    public Line(Product product, int qty) {
      if (product == null || qty <= 0) throw new IllegalArgumentException();
      this.product = product;
      this.qty = qty;
    }
    public Product product() { return product; }
    public int qty() { return qty; }
    public void add(int more) { if (more <= 0) throw new IllegalArgumentException();
qty += more; }
    public BigDecimal lineTotal() { return
product.price().multiply(BigDecimal.valueOf(qty)); }
    @Override public String toString() {
      return product.title() + " x" + qty + " = ₹" + lineTotal();
    }
  }

  private final List<Line> lines = new ArrayList<>();

  public void add(Product p, int qty) {
    // merge if same SKU already present
    for (Line l : lines) {
      if (l.product().sku().equals(p.sku())) { l.add(qty); return; }
    }
    lines.add(new Line(p, qty));
  }

  public BigDecimal total() {
    return lines.stream()
        .map(Line::lineTotal)
        .reduce(BigDecimal.ZERO, BigDecimal::add)
        .setScale(2, RoundingMode.HALF_UP);
  }

  @Override public String toString() {
    StringBuilder sb = new StringBuilder("Cart:\n");
    lines.forEach(l -> sb.append(" • ").append(l).append('\n'));
    sb.append("Total: ₹").append(total());
    return sb.toString();
```

```
  }

  public static void main(String[] args) {
    Cart cart = new Cart();
    Product p1 = new Product("SKU-101","Headphones", new
BigDecimal("1499.00"));
    Product p2 = new Product("SKU-202","Mouse",    new BigDecimal("599.50"));
    cart.add(p1, 2);
    cart.add(p2, 1);
    System.out.println(cart);
  }
}
```

Reasoning to interviewer:

- Product is a static nested class because it doesn't need access to the outer Cart; this avoids an accidental hidden reference and reduces memory pressure.
- The cart stays the aggregate owner of lines and totals; nested classes keep related concepts co-located and discoverable.
- Small nested value-types (like Size) live right beside the Product context for cohesion.

Future scope:

- Add discount policies at line/cart level.
- Persist Cart.Line rows with SKU and qty; compute prices server-side to prevent tampering.
- Use records for tiny immutable value-objects (Java 16+).

Scenario 64: Order with inner Item class, encapsulating item list inside Order

An order contains a list of items, encapsulated inside the Order class as an inner class. How does this work?

Answer:

Here I'll use a non-static inner class Item so each item automatically carries a reference to its owning Order. That lets Item access order metadata (like orderId, currency) without plumbing parameters around. The Order exposes methods to add/remove items; the item type itself remains hidden (encapsulation).

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.*;

class Order {
  private final String orderId;
  private final String currency;
  private final List<Item> items = new ArrayList<>();

  public Order(String orderId, String currency) {
    if (orderId == null || orderId.isBlank() || currency == null || currency.isBlank())
      throw new IllegalArgumentException("Invalid order");
    this.orderId = orderId.trim();
    this.currency = currency.trim();
  }

  // Non-static inner class: has implicit link to its Order.this
  public class Item {
    private final String sku;
    private final String title;
    private final BigDecimal unitPrice;
    private final int qty;

    private Item(String sku, String title, BigDecimal unitPrice, int qty) {
      if (sku == null || sku.isBlank() || title == null || title.isBlank()
        || unitPrice == null || unitPrice.signum() < 0 || qty <= 0)
        throw new IllegalArgumentException("Invalid item");
      this.sku = sku.trim();
      this.title = title.trim();
      this.unitPrice = unitPrice.setScale(2, RoundingMode.HALF_UP);
      this.qty = qty;
    }
```

```
    public BigDecimal lineTotal() { return unitPrice.multiply(BigDecimal.valueOf(qty));
}
    @Override public String toString() {
      return title + " x" + qty + " @ " + currency + " " + unitPrice + " = " + currency + " " +
lineTotal();
    }
    // Example of using outer state directly:
    public String itemTag() { return orderId + ":" + sku; }
  }

  public Item addItem(String sku, String title, BigDecimal price, int qty) {
    Item it = new Item(sku, title, price, qty);
    items.add(it);
    return it;
  }

  public BigDecimal total() {
    return items.stream()
            .map(Item::lineTotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add)
            .setScale(2, RoundingMode.HALF_UP);
  }

  @Override public String toString() {
    StringBuilder sb = new StringBuilder("Order ").append(orderId).append("\n");
    items.forEach(i -> sb.append(" • ").append(i).append('\n'));
    sb.append("Total: ").append(currency).append(" ").append(total());
    return sb.toString();
  }

  public static void main(String[] args) {
    Order o = new Order("ORD-1001", "INR");
    o.addItem("A1","Notebook", new BigDecimal("45.75"), 3);
    o.addItem("B2","Pen",     new BigDecimal("10.00"), 5);
    System.out.println(o);
  }
}
```

Reasoning to interviewer:

- A non-static inner Item can naturally read orderId and currency of its parent order-handy for tags, formatting, or validation rules tied to the order.
- The Item constructor is private, so only Order can create items. That encapsulates the invariant "items only exist inside an order."
- Clients don't manipulate the list directly; they go through addItem, keeping the aggregate consistent.

Future scope:

- Add domain rules (max items, currency consistency, stock checks) inside addItem.
- Expose a read-only view of items to callers (Collections.unmodifiableList).
- Emit domain events when items are added/removed for downstream services.

Scenario 65: Banking, Account with inner Transaction to record deposits/withdrawals

A banking app's Account class has an inner Transaction class to record deposits and withdrawals. Why use an inner class here?

Answer:

Transactions belong to one account and must capture the account's private state changes atomically (amount, type, timestamp, resulting balance). A private inner class Transaction can see Account's private fields and be constructed only by Account methods (deposit/withdraw). This cleanly models an event log without leaking mutators.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;
import java.util.*;

class Account {
  private final String accountNo;
  private BigDecimal balance = BigDecimal.ZERO.setScale(2,
RoundingMode.HALF_UP);
  private final List<Transaction> ledger = new ArrayList<>();

  public Account(String accountNo, BigDecimal opening) {
    if (accountNo == null || accountNo.isBlank() || opening == null || opening.signum()
< 0)
      throw new IllegalArgumentException("Invalid account");
    this.accountNo = accountNo.trim();
    deposit(opening); // records opening as a transaction
  }

  // Private inner class: only Account can create/modify
  private final class Transaction {
    final Instant time;
    final String type;      // "DEPOSIT" / "WITHDRAW"
    final BigDecimal amount;
    final BigDecimal resultingBalance;

    Transaction(String type, BigDecimal amount) {
      this.time = Instant.now();
      this.type = type;
      this.amount = amount.setScale(2, RoundingMode.HALF_UP);
```

```
        this.resultingBalance = balance; // reads outer's private field after mutation
      }

      @Override public String toString() {
        return time + " | " + type + " ₹" + amount + " | bal ₹" + resultingBalance;
      }
    }

    public synchronized void deposit(BigDecimal amt) {
      if (amt == null || amt.signum() <= 0) throw new IllegalArgumentException("Invalid
deposit");
      balance = balance.add(amt).setScale(2, RoundingMode.HALF_UP);
      ledger.add(new Transaction("DEPOSIT", amt));
    }

    public synchronized void withdraw(BigDecimal amt) {
      if (amt == null || amt.signum() <= 0) throw new IllegalArgumentException("Invalid
withdrawal");
      if (balance.compareTo(amt) < 0) throw new IllegalStateException("Insufficient
funds");
      balance = balance.subtract(amt).setScale(2, RoundingMode.HALF_UP);
      ledger.add(new Transaction("WITHDRAW", amt));
    }

    public BigDecimal getBalance() { return balance; }

    public List<String> statement() {
      List<String> out = new ArrayList<>();
      for (Transaction t : ledger) out.add(t.toString());
      return Collections.unmodifiableList(out);
    }

    public static void main(String[] args) {
      Account acc = new Account("SB-001", new BigDecimal("1000"));
      acc.deposit(new BigDecimal("250.50"));
      acc.withdraw(new BigDecimal("300"));
      System.out.println("Balance: ₹" + acc.getBalance());
      acc.statement().forEach(System.out::println);
    }
}
```

Reasoning to interviewer:

- The inner Transaction can read balance safely at the exact moment of mutation, perfect for a precise ledger entry.
- Because Transaction is private, external code can't forge or mutate entries; only Account creates them through domain methods.
- Synchronization around deposit/withdraw keeps ledger and balance consistent in multithreaded contexts.

Future scope:

- Make Transaction a public read-only projection (DTO) with no setters, or expose a mapper method.
- Persist the ledger as an append-only event store; rebuild balance from transactions if needed.
- Add transaction types (fees, interest), and support idempotency keys for safe retries.

Scenario 66: University, Department with inner Faculty linked only to that department

In a university system, the Department class has an inner Faculty class to represent professors linked only to that department. How can nested classes help?

Answer:

I'll model Department as the aggregate root. A professor (faculty) should logically "belong" to one department, so I'll use a non-static inner class Faculty. That way, each Faculty instance carries an implicit reference to its owning Department (via Department.this). Creation goes through a factory method on Department to keep invariants tight.

Code:

```
import java.util.*;

class Department {
   private final String code;
   private final String name;
   private final List<Faculty> facultyList = new ArrayList<>();

   public Department(String code, String name) {
      if (code == null || code.isBlank() || name == null || name.isBlank())
         throw new IllegalArgumentException("dept code/name required");
      this.code = code.trim();
      this.name = name.trim();
   }

   // Non-static inner class: bound to its Department
   public class Faculty {
      private final String professorId;
      private final String fullName;
      private final String specialization;

      private Faculty(String professorId, String fullName, String specialization) {
         if (professorId == null || professorId.isBlank() || fullName == null ||
fullName.isBlank())
            throw new IllegalArgumentException("prof details required");
         this.professorId = professorId.trim();
         this.fullName = fullName.trim();
         this.specialization = specialization == null ? "" : specialization.trim();
      }
```

```
    public String tag() { return Department.this.code + "-" + professorId; } // uses outer
state

    @Override public String toString() {
      return fullName + " (" + specialization + ") - " + Department.this.name;
    }
  }

  public Faculty addFaculty(String professorId, String fullName, String specialization)
{
    Faculty f = new Faculty(professorId, fullName, specialization);
    facultyList.add(f);
    return f;
  }

  public List<Faculty> getFaculty() { return List.copyOf(facultyList); }

  public static void main(String[] args) {
    Department cs = new Department("CSE", "Computer Science");
    cs.addFaculty("P101", "Dr. Mehta", "Distributed Systems");
    cs.addFaculty("P102", "Dr. Rao",   "Computer Vision");
    cs.getFaculty().forEach(System.out::println);
  }
}
```

Reasoning to interviewer:

- A non-static inner class “belongs” to its outer instance, which fits the domain: a professor is tied to exactly one department.
- Inner class can read department fields (e.g., code, name) without extra plumbing.
- Only Department can create Faculty (constructor is private), preserving consistency.

Future scope:

- Add role-based rules (HOD designation) within Department.
- Move hiring/removal policies into methods that audit changes.
- Expose a read-only DTO for Faculty to other modules to avoid leaking internals.

Scenario 67: Banking, Loan with inner EMI representing monthly instalments

In a banking system, the Loan class contains an inner class EMI that represents monthly installments. Why use an inner class here?

Answer:

Each EMI record is meaningful only in the context of its loan (it depends on principal, rate, tenure, remaining balance). I'll keep EMI as a private inner class so it can access Loan's private fields and be created only by the loan's amortization logic. The loan exposes a safe read-only statement.

Code:

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.*;

class Loan {
   private final String loanId;
   private final BigDecimal principal;
   private final BigDecimal annualRate; // e.g., 10% -> 0.10
   private final int tenureMonths;

   private final List<EMI> schedule = new ArrayList<>();

   public Loan(String loanId, BigDecimal principal, BigDecimal annualRate, int
tenureMonths) {
      if (loanId == null || loanId.isBlank() || principal == null || principal.signum() <= 0
         || annualRate == null || annualRate.signum() <= 0 || tenureMonths <= 0)
         throw new IllegalArgumentException("invalid loan");
      this.loanId = loanId.trim();
      this.principal = principal.setScale(2, RoundingMode.HALF_UP);
      this.annualRate = annualRate;
      this.tenureMonths = tenureMonths;
      buildSchedule();
   }

   private final class EMI {
      final int month;
      final BigDecimal principalComponent;
      final BigDecimal interestComponent;
      final BigDecimal remainingBalance;

      EMI(int month, BigDecimal p, BigDecimal i, BigDecimal rem) {
```

```
        this.month = month;
        this.principalComponent = p.setScale(2, RoundingMode.HALF_UP);
        this.interestComponent  = i.setScale(2, RoundingMode.HALF_UP);
        this.remainingBalance   = rem.setScale(2, RoundingMode.HALF_UP);
      }

      @Override public String toString() {
        BigDecimal amount = principalComponent.add(interestComponent);
        return "M" + month + " | EMI " + amount + " (P:" + principalComponent +
            ", I:" + interestComponent + ") | Bal:" + remainingBalance;
      }
    }

    private void buildSchedule() {
      BigDecimal monthlyRate = annualRate.divide(new BigDecimal("12"), 10,
RoundingMode.HALF_UP);
      // EMI formula: A = P * r * (1+r)^n / ((1+r)^n - 1)
      BigDecimal r = monthlyRate;
      BigDecimal onePlusRpowN = r.add(BigDecimal.ONE).pow(tenureMonths);
      BigDecimal emiAmt = principal.multiply(r).multiply(onePlusRpowN)
          .divide(onePlusRpowN.subtract(BigDecimal.ONE), 2,
RoundingMode.HALF_UP);

      BigDecimal balance = principal;
      for (int m = 1; m <= tenureMonths; m++) {
        BigDecimal interest = balance.multiply(r).setScale(2,
RoundingMode.HALF_UP);
        BigDecimal principalPart = emiAmt.subtract(interest).setScale(2,
RoundingMode.HALF_UP);
        balance = balance.subtract(principalPart).max(BigDecimal.ZERO);
        schedule.add(new EMI(m, principalPart, interest, balance));
      }
    }

    public List<String> statement() {
      List<String> out = new ArrayList<>();
      schedule.forEach(e -> out.add(e.toString()));
      return Collections.unmodifiableList(out);
    }

    public static void main(String[] args) {
      Loan loan = new Loan("LN-01", new BigDecimal("100000"), new
BigDecimal("0.12"), 12);
      loan.statement().forEach(System.out::println);
    }
}
```

Reasoning to interviewer:

- EMI needs intimate access to loan values and evolving balance-an inner class keeps that coupling intentional and safe.
- Only Loan can construct EMI entries, so third parties can't forge or alter the amortization trail.
- Consumers get a read-only statement, not mutable internals.

Future scope:

- Support prepayments and rebuild schedule from a certain month.
- Persist schedule as an event stream; recompute balances from transactions.
- Add hooks for changing rate (floating) with versioned EMIs.

Scenario 68: Shopping, Order with static nested OrderStatus (PLACED, SHIPPED, DELIVERED)

In a shopping app, the Order class has a static nested class OrderStatus (PLACED, SHIPPED, DELIVERED). How do nested classes organize this?

Answer:

OrderStatus is conceptually part of Order, but it doesn't need a reference to a specific order instance. A static nested enum keeps related types together without capturing outer state. It improves discoverability and namespacing (Order.Status.PLACED).

Code:

```
class Order {
   public enum Status { PLACED, SHIPPED, DELIVERED, CANCELLED }

   private final String orderId;
   private Status status = Status.PLACED;

   public Order(String orderId) {
      if (orderId == null || orderId.isBlank()) throw new IllegalArgumentException("id
required");
      this.orderId = orderId.trim();
   }

   public void ship() {
      if (status != Status.PLACED) throw new IllegalStateException("Only placed orders
can be shipped");
      status = Status.SHIPPED;
   }

   public void deliver() {
      if (status != Status.SHIPPED) throw new IllegalStateException("Ship before
deliver");
      status = Status.DELIVERED;
   }

   public Status status() { return status; }

   @Override public String toString() { return orderId + " - " + status; }

   public static void main(String[] args) {
      Order o = new Order("ORD-42");
      System.out.println(o);
      o.ship();
      System.out.println(o);
```

```
        o.deliver();
        System.out.println(o);
    }
}
```

Reasoning to interviewer:

- A static nested enum groups status values inside Order's namespace without an outer reference, keeping the API tidy (Order.Status).
- Enforcing transitions in methods (ship(), deliver()) protects invariants, clients can't jump from PLACED to DELIVERED directly.
- Enum gives type safety over string statuses and supports switch statements cleanly.

Future scope:

- Add timestamps for each transition; store a mini state machine.
- Introduce intermediate statuses (PACKED, OUT_FOR_DELIVERY, RETURNED).
- Expose an event when status changes so fulfillment and notifications can react.

Interfaces

Scenario 69: Payment app, enforcing standardization with a Payment interface

In a payment app, multiple classes (UPIPayment, CardPayment) must implement a common Payment interface. How do interfaces enforce standardization?

Answer:

In a payment ecosystem, every payment mode (UPI, Card, Wallet, NetBanking) must follow the same contract: validate → authorize → capture/refund. Instead of duplicating rules, I define a Payment interface that enforces a standard set of methods. Each class implements its logic but clients always talk to the interface, not the concrete type.

Code:

```
import java.math.BigDecimal;

interface Payment {
   boolean validate();
   boolean pay(BigDecimal amount);
   boolean refund(BigDecimal amount);
}

class UPIPayment implements Payment {
   private final String vpa; // e.g. rajat@okicici
   public UPIPayment(String vpa) { this.vpa = vpa; }

   @Override public boolean validate() {
      System.out.println("Validating UPI VPA: " + vpa);
      return vpa.contains("@");
   }
   @Override public boolean pay(BigDecimal amount) {
      System.out.println("UPI payment of ₹" + amount + " successful.");
      return true;
   }
   @Override public boolean refund(BigDecimal amount) {
      System.out.println("Refunded ₹" + amount + " via UPI.");
      return true;
   }
}

class CardPayment implements Payment {
   private final String cardToken; // PCI-safe token
```

```
  public CardPayment(String token) { this.cardToken = token; }

  @Override public boolean validate() {
    System.out.println("Validating Card Token: " + cardToken);
    return cardToken.startsWith("tok_");
  }
  @Override public boolean pay(BigDecimal amount) {
    System.out.println("Card charged ₹" + amount);
    return true;
  }
  @Override public boolean refund(BigDecimal amount) {
    System.out.println("Refunded ₹" + amount + " to Card.");
    return true;
  }
}

class Checkout {
  public static void process(Payment p, BigDecimal amt) {
    if (!p.validate()) throw new IllegalStateException("Invalid instrument");
    if (p.pay(amt)) System.out.println("Payment success.\n");
  }

  public static void main(String[] args) {
    process(new UPIPayment("rajat@okhdfcbank"), new BigDecimal("599"));
    process(new CardPayment("tok_visa123"), new BigDecimal("1299"));
  }
}
```

Reasoning to interviewer:

- The Payment interface enforces a standard contract - all payment classes must implement the same methods.
- Client code (Checkout) is agnostic, it calls validate() and pay() without knowing if it's UPI or Card.
- This makes the system open for extension: adding WalletPayment or CryptoPayment just means writing a new class that implements Payment, no need to change Checkout.

Future scope:

- Add default methods like generateReceipt() in the interface.
- Use dependency injection to inject payment types dynamically.
- Extend interface with async operations for retry/failure handling.

Scenario 70: Sports tournament, extensibility via a GameRules interface

A sports tournament requires different games (Cricket, Football, Chess) to implement a GameRules interface. How does this help in extensibility?

Answer:

Every sport has its own rules: overs in cricket, halves in football, time controls in chess. But the tournament organizer only cares that each game can start, end, and declare a winner. By defining a GameRules interface, I enforce a contract: all games must expose the same high-level lifecycle methods.

Code:

```
interface GameRules {
   void startMatch();
   void play();
   String declareWinner();
}

class Cricket implements GameRules {
   @Override public void startMatch() { System.out.println("Cricket match begins with
toss!"); }
   @Override public void play()      { System.out.println("Playing 20 overs each side."); }
   @Override public String declareWinner() { return "Team batting second wins by 5
wickets"; }
}

class Football implements GameRules {
   @Override public void startMatch() { System.out.println("Football match kicks
off!"); }
   @Override public void play()      { System.out.println("Two halves of 45 minutes
each."); }
   @Override public String declareWinner() { return "Team A wins 2-1"; }
}

class Chess implements GameRules {
   @Override public void startMatch() { System.out.println("Chess game starts with
white's move."); }
   @Override public void play()      { System.out.println("Players alternate moves with
time control."); }
   @Override public String declareWinner() { return "Black wins by checkmate"; }
}

class Tournament {
```

```
    public static void main(String[] args) {
        GameRules[] games = { new Cricket(), new Football(), new Chess() };
        for (GameRules g : games) {
            g.startMatch();
            g.play();
            System.out.println("Result: " + g.declareWinner());
            System.out.println();
        }
    }
}
```

Reasoning to interviewer:

- The GameRules interface defines the lifecycle contract, every game must start, play, and declare a winner.
- Polymorphism: the tournament doesn't need to know the exact game type, it just runs g.play().
- This makes the system extensible, tomorrow we can add Hockey or Tennis simply by implementing GameRules.

Future scope:

- Add default methods like showRules() for common instructions.
- Support multi-round tournaments by wrapping GameRules inside a Match object with scheduling logic.
- Integrate scoring APIs so results feed into leaderboards automatically.

Object Class Methods

Scenario 71: Student management app, overriding toString() for readable info

In a student management app, when printing student objects, they show memory references. How will you override toString() to display readable info?

Answer:

By default, Java's toString() prints something like Student@5a07e868, which is not meaningful. For user-facing readability, I'll override toString() to return a nicely formatted string with important fields like name, roll number, and department.

Code:

```
class Student {
   private final String name;
   private final int rollNo;
   private final String department;

   public Student(String name, int rollNo, String department) {
      if (name == null || name.isBlank() || rollNo <= 0 || department == null ||
department.isBlank())
         throw new IllegalArgumentException("Invalid student data");
      this.name = name.trim();
      this.rollNo = rollNo;
      this.department = department.trim();
   }

   @Override
   public String toString() {
      return "Student{name='" + name + "', rollNo=" + rollNo + ", department='" +
department + "'}";
   }

   public static void main(String[] args) {
      Student s1 = new Student("Riya", 101, "Computer Science");
      Student s2 = new Student("Arjun", 102, "Mechanical Engg");
      System.out.println(s1);  // prints clean readable info
      System.out.println(s2);
   }
}
```

Reasoning to interviewer:

- Overriding toString() improves debugging and logging, logs show real student info instead of raw memory references.
- This also helps when student objects are displayed in lists or UIs (JTable, REST responses).
- It's good practice to override toString() for all domain objects that may appear in logs or reports.

Future scope:

- Customize formatting using String.format() or JSON style ({"name":"Riya",...}) for easy integration with APIs.
- In large systems, use Lombok's @ToString or JSON libraries for automatic formatting.

Scenario 72: Shopping system, overriding equals() and hashCode() for product identity

In a shopping system, two products with the same ID should be considered equal. How will you override equals() and hashCode()?

Answer:

In an e-commerce system, two Product objects with the same ID should be treated as the same logical product, even if created separately in memory. By default, equals() checks memory references (==). To ensure correct behavior in collections (HashSet, HashMap), I'll override both equals() and hashCode().

Code:

```
import java.util.Objects;

class Product {
   private final String id;
   private final String name;
   private final double price;

   public Product(String id, String name, double price) {
      if (id == null || id.isBlank() || name == null || name.isBlank() || price < 0)
         throw new IllegalArgumentException("Invalid product");
      this.id = id.trim();
      this.name = name.trim();
      this.price = price;
   }

   // Products are equal if they share the same ID
   @Override
   public boolean equals(Object o) {
      if (this == o) return true;              // same reference
      if (!(o instanceof Product)) return false;  // type check
      Product other = (Product) o;
      return id.equals(other.id);              // equality based on product ID
   }

   @Override
   public int hashCode() {
      return Objects.hash(id); // consistent with equals
   }

   @Override
   public String toString() {
```

```
    return "Product{id='" + id + "', name='" + name + "', price=" + price + "}";
  }

  public static void main(String[] args) {
    Product p1 = new Product("P101", "Laptop", 55000);
    Product p2 = new Product("P101", "Laptop - 2025 Edition", 56000);
    Product p3 = new Product("P202", "Phone", 30000);

    System.out.println("p1 equals p2? " + p1.equals(p2)); // true (same ID)
    System.out.println("p1 equals p3? " + p1.equals(p3)); // false

    java.util.Set<Product> set = new java.util.HashSet<>();
    set.add(p1);
    set.add(p2); // won't be added again, since ID is same
    set.add(p3);
    System.out.println("Set size: " + set.size()); // 2
  }
}
```

Reasoning to interviewer:

- equals() ensures logical equality (based on ID), not just memory equality.
- hashCode() must be consistent with equals(); otherwise, HashSet or HashMap behave incorrectly.
- This makes sure collections treat p1 and p2 as the same product if their IDs match.

Future scope:

- If product identity is composite (like id + variant + seller), extend equals() and hashCode() accordingly.
- Use record (Java 16+) for immutable value objects with automatic equals(), hashCode(), and toString().

Relationships (Association, Aggregation, Composition, Dependency Injection)

Scenario 73: Aggregation vs Composition, Department↔Teachers (aggregation), Car→Engine (composition)

In a school system, a Department has many Teachers (aggregation), while a Car has an Engine (composition). How do you model both?

Answer:

- **Aggregation:** a Department "has many" Teachers, but teachers can exist without the department (they can transfer). Keep a list of Teacher references; don't control their lifecycle.
- **Composition:** a Car "has an" Engine that cannot live independently. The Car creates and owns the Engine; when the car is gone, so is the engine.

Code (aggregation + composition):

```
import java.util.*;

class Teacher {
   private final String id;
   private final String name;
   public Teacher(String id, String name) {
      if (id == null || id.isBlank() || name == null || name.isBlank())
         throw new IllegalArgumentException();
      this.id = id.trim(); this.name = name.trim();
   }
   @Override public String toString() { return name + " [" + id + "]"; }
}

// Aggregation: Department refers to Teachers but doesn't own their lifecycle
class Department {
   private final String code;
   private final List<Teacher> teachers = new ArrayList<>();
   public Department(String code) { this.code = code; }
   public void addTeacher(Teacher t) { teachers.add(t); }    // just holds a reference
   public List<Teacher> getTeachers() { return List.copyOf(teachers); }
}

// Composition: Engine is created and owned by Car; no public constructor for Engine
class Car {
   private final Engine engine;   // strong ownership
   public Car(int displacementCc) { this.engine = new Engine(displacementCc); }
   public int horsepower() { return engine.estimateHP(); }
```

```
  private static final class Engine {            // not reusable outside Car
    private final int displacementCc;
    private Engine(int displacementCc) {
      if (displacementCc <= 0) throw new IllegalArgumentException();
      this.displacementCc = displacementCc;
    }
    int estimateHP() { return (int)(displacementCc * 0.12); }
  }
}

class DemoAggComp {
  public static void main(String[] args) {
    Teacher t1 = new Teacher("T01","Meera");     // exists independently
    Teacher t2 = new Teacher("T02","Raghav");

    Department cse = new Department("CSE");
    cse.addTeacher(t1);
    cse.addTeacher(t2);
    System.out.println(cse.getTeachers());      // aggregation

    Car car = new Car(1500);
    System.out.println("HP: " + car.horsepower()); // composition
  }
}
```

Reasoning to interviewer:

- **Aggregation =** loose “has-a” relationship; lifecycle is independent.
- **Composition =** strong ownership; the part cannot outlive the whole.
- I used a private static final Engine to make creation and lifecycle strictly inside Car.

Future scope:

- **For aggregation,** store only teacherIds to decouple services; fetch details from a repository.
- **For composition,** expose read-only snapshots of engine data if needed, not the engine itself.

Scenario 74: Association (many-to-many), Doctor↔Patient

In a hospital, a Doctor can treat many Patients, and a Patient can consult many Doctors. How do you show this association in OOP?

Answer:

A doctor can treat many patients; a patient can consult many doctors. Model a bidirectional association with sets on both sides and helper methods that keep the relationship in sync.

Code (many-to-many association):

```
import java.util.*;

class Patient {
   private final String id;
   private final String name;
   private final Set<Doctor> doctors = new HashSet<>();

   public Patient(String id, String name) {
      this.id = id; this.name = name;
   }
   public void addDoctor(Doctor d) {
      if (d == null) return;
      if (doctors.add(d)) {
         d.addPatientInternal(this);  // keep both sides consistent
      }
   }
   void addDoctorInternal(Doctor d) { doctors.add(d); } // package/private helper

   @Override public String toString() { return "Patient(" + name + ")"; }
   public Set<Doctor> getDoctors() { return Set.copyOf(doctors); }
}

class Doctor {
   private final String id;
   private final String name;
   private final Set<Patient> patients = new HashSet<>();

   public Doctor(String id, String name) {
      this.id = id; this.name = name;
   }
   public void addPatient(Patient p) {
      if (p == null) return;
      if (patients.add(p)) {
         p.addDoctorInternal(this);   // mirror link
```

```
    }
  }
  void addPatientInternal(Patient p) { patients.add(p); }

  @Override public String toString() { return "Doctor(" + name + ")"; }
  public Set<Patient> getPatients() { return Set.copyOf(patients); }
}

class DemoAssoc {
  public static void main(String[] args) {
    Doctor d1 = new Doctor("D1","Dr. Mehta");
    Doctor d2 = new Doctor("D2","Dr. Kapoor");
    Patient p1 = new Patient("P1","Riya");
    Patient p2 = new Patient("P2","Arjun");

    d1.addPatient(p1);
    d1.addPatient(p2);
    d2.addPatient(p1);

    System.out.println(d1 + " -> " + d1.getPatients());
    System.out.println(d2 + " -> " + d2.getPatients());
    System.out.println(p1 + " -> " + p1.getDoctors());
    System.out.println(p2 + " -> " + p2.getDoctors());
  }
}
```

Reasoning to interviewer:

- I used sets to avoid duplicates and synchronizing helpers to keep both sides in lockstep.
- This keeps domain code clean: calling one method maintains the association consistently.
- In a DB-backed system, this maps naturally to a join table doctor_patient(doctor_id, patient_id).

Future scope:

- Replace direct references with IDs and a repository layer for lazy loading.
- Add an association entity Consultation carrying date, diagnosis, notes (turning many-to-many into two one-to-many relations via the join entity).

Scenario 75: Dependency Injection, UserService depends on DatabaseService

In a web app, a UserService depends on a DatabaseService. Instead of creating it inside, how will you inject it externally?

Answer:

Instead of new DatabaseService() inside UserService, accept it via the constructor (constructor injection). This decouples components, improves testability (pass a mock), and supports swapping implementations (e.g., PostgreSQL vs in-memory) without changing UserService.

Code (interface + constructor injection):

```
import java.util.*;

interface DatabaseService {
   Optional<String> findUserEmail(String userId);
   void saveUser(String userId, String email);
}

class InMemoryDatabase implements DatabaseService {
   private final Map<String,String> store = new HashMap<>();
   @Override public Optional<String> findUserEmail(String userId) { return
Optional.ofNullable(store.get(userId)); }
   @Override public void saveUser(String userId, String email) { store.put(userId,
email); }
}

class UserService {
   private final DatabaseService db;  // dependency

   // Inject externally
   public UserService(DatabaseService db) {
      if (db == null) throw new IllegalArgumentException("db required");
      this.db = db;
   }

   public void register(String userId, String email) {
      if (db.findUserEmail(userId).isPresent()) throw new IllegalStateException("User
exists");
      db.saveUser(userId, email);
   }

   public Optional<String> emailOf(String userId) {
```

```
    return db.findUserEmail(userId);
  }
}

class DemoDI {
  public static void main(String[] args) {
    DatabaseService db = new InMemoryDatabase();   // could be a real DB
implementation
    UserService users = new UserService(db);      // injected

    users.register("u1", "riya@example.com");
    System.out.println(users.emailOf("u1").orElse("not found"));
  }
}
```

Reasoning to interviewer:

- Constructor injection makes the dependency explicit and prevents a half-initialized service.
- UserService depends on the DatabaseService interface, not a concrete class → loose coupling.
- Unit tests can pass a fake/mock DatabaseService to test UserService logic in isolation.

Future scope:

- Switch to a DI container (Spring) for lifecycle and scoping, but keep the same constructor contract.
- Add more interfaces (CacheService) and compose them without changing UserService's core logic.
- Use method/field injection only when unavoidable; constructor injection is the most robust for required dependencies.